

Evaluating Alias Tables for Weighted Random Sampling on GPUs

Bachelor's Thesis of

Karl Sassie

At the Department of Informatics
Institute for Visualization and Data Analysis (IVD)

Reviewer: Prof. Dr.-Ing. Carsten Dachsbacher
Advisor: M.Sc. Lucas Alber

Duration: 12.01.2024 – 03.31.2025

I declare that I have developed and written the enclosed thesis completely by myself, and have not used sources or means without declaration in the text.

Karlsruhe, 03.31.2025

.....
(Karl Sassie)

Abstract

Weighted Random Sampling (WRS) is fundamental in simulations, Monte Carlo methods, and other computational tasks requiring random selection according to discrete distributions. This thesis evaluates Alias Tables, an efficient data structure for constant-time weighted sampling, focusing specifically on parallel GPU implementations. We analyze the Parallel Splitting-based Alias Table Construction (PSA) algorithm, emphasizing its applicability and efficiency on GPU architectures. Specifically, we evaluate whether constructing an Alias Table using PSA and subsequently sampling from it provides performance benefits compared to simpler methods like inverse transform sampling, which require less construction overhead. Our Vulkan-based implementation, which is portable across different GPU platforms, demonstrates that the Alias Table method using PSA offers substantial performance advantages only when the number of samples significantly exceeds the distribution size, making it particularly suitable for applications requiring extensive sampling from dynamically generated distributions on GPUs.

Acknowledgments

I want to thank Prof. Dr.-Ing. Carsten Dachsbacher for reviewing this thesis. I thank Lucas Albert for suggesting the interesting topic of weighted random sampling and offering great advice in research directions and time planning. Furthermore, I am also grateful to Hübschle-Schneider and Sanders and Lehmann *et al.* for publishing their source code of their PSA implementation, which was a great resource to get started with the alias method. Last but not least, many thanks to my good friend Jakob Schmid, who always had an open ear and helped me to organize my thoughts while implementing and writing the thesis.

Contents

1	Introduction	1
2	Background	3
2.1	Weighted Random Sampling	3
2.2	GPU Architectures	3
2.3	Forward progress	5
2.3.1	Subgroup forward progress	5
2.3.2	Inter-workgroup	5
2.4	Pseudo Random Number Generators	6
2.4.1	Counter-based PRNG	6
2.4.2	Uniform Floating-Point Numbers	7
2.4.3	Uniform Integer	7
2.4.3.1	Fast Dice Roller	8
3	Related Work	9
3.1	Inverse transform sampling	9
3.2	Cutpoint method	9
3.3	Alias Table	10
3.3.1	Sequential Construction	10
3.3.2	Parallel Construction	11
3.4	Scan	14
3.4.1	Register scan	14
3.4.2	Subgroup scan	14
3.4.3	Ranked	15
3.4.4	Raking	15
3.4.5	Arrays of arbitrary size	16
3.4.5.1	Single pass approach	16
4	Parallel Scan	19
4.1	Subgroup scan	19
4.2	Block Scan	19
4.3	Inter block scan	20
4.3.1	Single dispatch scan	21
4.3.1.1	Parallel look-back	21
4.3.1.2	Proving correctness	21
4.4	Scan derivatives	23
4.5	Numerical stability	24
4.6	Uncompetitive methods	25
5	Inverse transform sampling	27
5.1	Cutpoint method	27
5.1.1	Uniform sampling	28
6	Alias method	29
6.1	Parallel splitting based alias table construction (PSA)	29

6.1.1	Splitting	30
6.1.2	Packing	30
6.1.2.1	Uncompetitive Methods	32
6.1.3	Split-Packing	32
6.2	Sampling	32
6.2.1	Uniformly select an entry	33
7	Evaluation	35
7.1	Memory bandwidth baseline	36
7.2	Prefix Sum	36
7.2.1	Prefix over Partitions	38
7.2.2	Numerical stability	39
7.3	Inverse Transform Sampling	39
7.3.1	Accuracy	41
7.4	Alias method	42
7.4.1	Parallel splitting based alias table construction	42
7.4.1.1	Splitting	42
7.4.1.2	Packing	43
7.4.1.3	Split-Packing	45
7.4.1.4	Complete construction	48
7.4.2	Sampling	49
7.5	Comparison	51
8	Conclusion	55
	Bibliography	57

1. Introduction

Drawing random samples from a population, given a discrete probability distribution, is crucial in various fields of computer science, including simulations, random graph walks (i.e. Page Ranking), Monte Carlo methods, and many more. As the complexity of such applications grows and single-core clock speeds plateau, the potential for improvement increasingly lies in parallel algorithms. The Alias Table, a data structure enabling efficient sampling of random variables in $\mathcal{O}(1)$ time, is one solution. While sequential algorithms for Alias Table construction are well established [Wal77; Vos91], parallel methods such as Parallel Splitting-based Alias Table Construction (PSA) [HS22] are less known, and only a few open-source implementations have been developed. The only open-source implementation known to us uses CUDA, which is only available on NVIDIA GPUs and additionally does not expose the fixed-function pipelines, such as rasterization. In computer graphics, fixed-function pipelines and cross-platform compatibility are often needed, which makes the integration of CUDA implementations difficult. Therefore, a portable and efficient implementation of PSA is needed to enable cross-platform sampling suitable for diverse applications and hardware architectures.

Structure: In chapter 2, we shortly introduce fundamental concepts, such as a formal definition of weighted random sampling and pseudo number generators, an abstract model of GPU architectures and, lastly, a short introduction to forward progress guarantees found in most GPUs today. Chapter 3 introduces the `alias-method` and the PSA algorithm. We also discuss the inverse-transform sampling algorithm, which we use as a baseline for comparisons. Since our sampling algorithms need an efficient parallel `scan` operation, we additionally summarize approaches used in open source libraries. In chapters 4, 5 and 6 we cover our implementations. In chapter 7 we first evaluate our implementations in isolation followed by a comparison. Specifically, we compare our sampling methods, while considering both sampling and construction time and determine the best method for a given distribution size and sample size.

Source-Code: The complete source code of our implementation and evaluation is publicly available on Github (<https://github.com/kistenklaus/merian-wrs.git>).

2. Background

2.1 Weighted Random Sampling

Weighted Random Sampling (WRS) is the process of selecting random items from a dataset, where each item’s probability of selection is proportional to a given weight. Unlike uniform random sampling, where all elements have an equal chance of being selected, WRS ensures that elements with higher weights are chosen more frequently.

Given a set of n elements $x_1, x_2, \dots, x_n$ with associated non-negative weights $\Omega = \{\omega_1, \omega_2, \dots, \omega_n\}$, the probability of selecting an element x_i is:

$$P[X = x_i] = \frac{\omega_i}{\sum_{k=1}^n \omega_k}$$

where $\sum_{k=1}^n \omega_k$ is the total weight of all elements.

We further define σ as the total weight and $\bar{\omega}$ as the average weight. Table 2.1 gives an overview of commonly used symbols and their definitions.

Symbol	Definition	Description
Ω		Set of all weights
n	$ \Omega $	Amount of elements
σ	$\sum_{k=1}^n \omega_k$	Total weight
$\bar{\omega}$	$\frac{\sigma}{n}$	Average weight

Table 2.1: Summary of commonly used symbols.

2.2 GPU Architectures

GPUs are massively parallel machines capable of executing thousands of tasks concurrently, distributed across multiple SIMD units. On NVIDIA platforms, a group of 32 threads is known as a warp, whereas AMD refers to similar groups as wavefronts, with some machines even supporting multiple wavefront sizes. To maintain consistency, we will use Vulkan

terminology, referring to such groups as **subgroups**, and the individual units within as **invocations**. This execution style, where a single instruction operates across multiple threads, is termed SIMT (Single Instruction, Multiple Threads). The processors executing SIMT instructions are called **SIMT-processors**, referred to as Streaming Multiprocessors (SMs) by NVIDIA and Compute Units (CUs) by AMD.

Like Intel’s hyper-threading in CPUs, most SIMT processors handle multiple instruction streams (subgroups) concurrently, allowing for efficient context switches by maintaining distinct register sets for each stream. This design enables the processor to switch to another instruction stream while waiting on long-latency operations, thereby maintaining high instruction throughput. Internally, SIMT processors may even duplicate certain pipeline stages, such as the Floating Point Unit (FPU), to achieve true parallelism for some instruction types. Furthermore, subgroups are organized into workgroups, allowing for communication and synchronization through shared memory and execution barriers. Modern GPUs typically contain between 5 and 200 SIMT-processors, enabling them to process tens of thousands of tasks simultaneously.

The limit to this parallelism is often defined by the number of subgroups a SIMT-processor can handle concurrently, or in GPU terminology; the maximum number of occupant subgroups, which depends on the required registers and the shared memory allocated per workgroup. Additionally, the hardware sets an upper limit on the number of subgroups and workgroups that can be active simultaneously. **Occupancy** is defined as the percentage of active subgroups relative to the maximum supported by the hardware.

GPU memory architectures differ significantly from those in CPUs, which typically optimize for latency. In contrast, GPUs are designed to maximize memory throughput, which inherently results in higher latency. Efficient GPU programming techniques, such as achieving high occupancy, help mitigate the impact of this latency by hiding long memory accesses behind instructions from other active subgroups. Another critical concept is memory coalescing, where Load Store Units (LSUs) at different levels of the memory hierarchy (e.g., L1 cache, L2 cache, and VRAM) bundle memory accesses into transactions. These transactions typically range from 128 to 256 bytes. While the rate of memory transactions that an LSU can handle per second is limited and significantly lower than the number of memory accesses that can be issued, aligned and contiguous accesses by invocations within a subgroup can be combined into a single transaction. We call this **memory-coalescing**.

When utilizing shared memory, consideration of the underlying hardware design is crucial. For example, NVIDIA implements a unified L1 and shared memory, meaning that if compute workloads require more shared memory it implicitly reduces the size of the L1-cache. To our knowledge, other manufacturers implement them separately, where the shared memory regions can still be used as a software-controlled cache. Regardless of the specific implementation details, all GPUs known to us divide the shared memory into banks of 4 bytes each. Memory addresses map to these banks such that an address a corresponds to

$$\text{bank}(a) = \frac{a}{4} \bmod (\text{amount-of-banks})$$

When x invocations access different addresses in the same bank, the accesses are serialized, leading to what is known as an x -way **bank conflict**.

Building on our discussion of GPU architectures, we now introduce additional terminology that is essential for understanding how the **host** (CPU) and the **device** (GPU) interact through the Vulkan API. Vulkan is a comprehensive graphics and compute API that acts as an interface between these components. It enables the execution of specialized programs known as shaders, particularly compute shaders, that handle data-parallel computation

tasks efficiently on the GPU. One key operation is a `dispatch`, a command issued by the host to start shader executions on the device, specifying the number of workgroups involved. Additionally, `pipeline-barriers` are synchronization primitives within Vulkan, ensuring proper sequencing and synchronization between dispatches to prevent data races and ensure orderly execution of commands.

2.3 Forward progress

2.3.1 Subgroup forward progress

Intra-subgroup progress is a property of a GPU’s SIMT scheduler, which defines how the execution of invocations within a subgroup is guaranteed to progress.

In a typical SIMT execution model, all invocations within a subgroup execute instructions in lockstep. When control flow diverges, inactive threads are masked out via predication, and both branches (e.g., if and else) are executed sequentially.

Importantly, no assumptions can be made on which branch is executed first, this may lead to deadlocks if one branch depends on the outcome of the other branch (e.g. mutexes with atomic compare exchange, where the else branch waits on the results of the if branch). Additionally, GPU schedulers typically attempt to reconverge divergent branches as early as possible, which can further increase the likelihood of such deadlocks.

NVIDIA’s independent thread scheduler [Cor17] partially addresses these limitations by implementing a fair scheduler that does not prioritize optimal reconvergence. However, this scheduler still falls short of the concurrent forward progress guarantees found in CPU architectures. Livelocks remain possible (e.g., in scenarios such as the dining philosopher problem), and there is no strict guarantee that any thread will take an initial step. This last property is most likely an implication of the forward progress guarantees provided by the workgroup scheduler.

Due to the variability in subgroup progress guarantees across hardware, Vulkan does not define specific forward progress guarantees. Instead, it provides a set of subgroup intrinsics, including barriers, atomic operations, shuffles, and broadcasts, which enable limited but well-defined communication and synchronization in any progress model. These intrinsics allow parallel algorithms to achieve portability without requiring precise knowledge of intra-subgroup progress guarantees. In addition, they often yield performance advantages by closely aligning with hardware-level optimizations.

2.3.2 Inter-workgroup

Inter-workgroup synchronization was previously avoided due to the unclear forward progress guarantees provided by the workgroup scheduler.

Traditional approaches use multiple dispatches to coordinate. By submitting dispatches together and synchronizing with pipeline barriers, it may be possible to mask the decode and dispatch latencies; however, such approaches enforce a serial dependency between dispatches, which may involve redundant memory accesses and reduce temporal cache locality.

In recent years, several studies [Dua16; GSO12] have explored methods to exploit undefined behaviors in GPU scheduling to improve inter-workgroup performance. These methods can often lead to substantial performance gains, although the speed-up often varies significantly between hardware architectures, where some architectures may also show a negative speed-up [SDB+16].

Although manufacturers rarely specify a forward progress model, empirical studies [SDB+16] indicate that most modern architectures provide a forward progress guarantee known as **Occupancy Bound Execution (OBE)**. OBE states that if a workgroup has made an initial step, it is guaranteed to be scheduled fairly afterwards.

We can intuitively understand OBE by looking at an example scheduler. Assume that we implement a job system which executes tasks with T threads and does not implement any way to preempt running tasks. Initially, this scheduler will take T tasks and execute them. Once all T tasks are running, the job system will have to wait for one of the tasks to finish to schedule the next job. The C++ standard defines this forward progress guarantee as `parallel forward progress`¹.

OBE is very similar to this forward progress guarantee, with the only modification that the amount of threads T is dependent on the maximum amount of workgroups, which is dependent on the occupancy of a program.

Further research [SSR+21] suggests that most modern architectures do not support a **Linear Occupancy Bound Execution (LOBE)** forward progress guarantee, which would extend OBE by ensuring that any workgroup with a lower ID than a workgroup that has made a step is also scheduled fairly. This forward guarantee would be analogous to a job system, where tasks that are submitted first are also executed first.

2.4 Pseudo Random Number Generators

Pseudo Random Number Generators (PRNGs) are a class of algorithms that produce sequences of seemingly random numbers in a deterministic manner, entirely determined by an initial key.

We can define all PRNGs by their state space $\mathcal{S}$, their state transition function $f: \mathcal{S} \rightarrow \mathcal{S}$, their output function $g: \mathcal{S} \rightarrow \mathcal{U}$ and their output space $\mathcal{U}$.

The process to select a pseudo random number $u \in \mathcal{U}$, in state $s_n \in \mathcal{S}$, now involves transitioning the state with $s_{n+1} = f(s_n)$ and drawing the sample $u = g(s_{n+1})$.

Conventional PRNGs, such as the Mersenne-Twister, Linear congruential generators (LCGs) or Linear-feedback shift registers (LFSRs) are initialized with a key, which determines the initial state $s_0 \in \mathcal{S}$; random samples are then drawn by sequentially applying the transition function f , which yields the next state, which is then often used with a trivial output function to generate new pseudo random number. This inherently sequential execution does not map very well to massively parallel machines. A common solution is to use a single PRNG per thread. However, some PRNGs, especially those that yield high-quality samples, have large states (e.g., Mersenne-Twister has a state of around $2.4Kb$), which is problematic as memory close to the processor (i.e. fast, often cached memory) is a very limited resource.

Mersenne-Twister engines and similar large-state PRNGs are therefore unsuitable on GPUs, except when very long periods are needed (above 2^{128}).

2.4.1 Counter-based PRNG

Salmon *et al.* introduces Counter-based PRNGs, which are better suited for a parallel execution model, as they avoid the serial dependency between consecutive states. Instead

¹“If a thread offers parallel forward progress guarantee, the implementation is not required to ensure that the thread will eventually make progress if it has not yet executed any execution step (I/O, volatile, atomic, or synchronization), but once this thread has executed a step, it provides concurrent forward progress guarantees (this rule describes a thread in a thread pool that executes tasks in arbitrary order).” - cppreference

of sequentially dependent state transitions, counter-based PRNGs define state s_n without a dependency on the previous state s_{n-1} . In the simplest case, the state is merely a counter, giving this subgroup of PRNGs its name. These PRNGs very nicely map to parallel execution models, because computing the state s_n is now possible without the previous state, which means that drawing the first random number is as simple as drawing the n th random number. The process to draw a pseudo random number now involves a more complex output function. Most counter-based PRNGs known to us use a pseudo random bijection $b_k(x)$, where k is the key used to initialize the PRNG. Pseudo random bisections or permutations are commonly found in cryptography with block ciphers. On CPUs this is especially beneficial as most modern architectures implement dedicated hardware to compute such bijections efficiently. On GPUs, we sadly do not have this luxury. Philox is a counter-based PRNG that repeatedly multiplies two integers, utilizing high and low bits in a Feistel structure. These operations are bijective and inexpensive, resulting in an efficient PRNG well-suited for GPUs [SMDS11].

2.4.2 Uniform Floating-Point Numbers

Because PRNGs generally only generate pseudo random bit strings, we want to shortly introduce a common solution to map pseudo random bit strings to single precision floating point numbers between 0 and 1 (exclusive). Single precision floating point numbers, as defined in IEEE-754, contain 8 bits for the exponent and 23 bits for the mantissa. If we ignore negative and de-normalized numbers, a floating point number with exponent E and mantissa M represents the value $2^{127-E}(1 + \frac{M}{2^{23}})$. If we set $E = 127$ and fill M with 23 random bits, we end up with a pseudo random float uniformly distributed in $[1, 2)$. After subtracting one we end up with a pseudo random float between $[0, 1)$. This method is fairly efficient and, more importantly, ensures that the resulting floating point number is sampled correctly.

2.4.3 Uniform Integer

Another very common problem is uniformly selecting an integer below an upper exclusive bound $U \in \mathbb{N}_+$. If U is a power of two, this is trivial. In all other cases, the problem is surprisingly difficult. A naive implementation generates a random floating-point number $u \in [0, 1)$ and then multiplies it with U and floors the result. Although this solution is very efficient, it is also biased. Specifically, this introduces a bias because it uses a floating-point number, with a fixed-size mantissa (i.e. 23bits), which means that if $\log_2 U > 23$ or if the amount of possible values of the floating-point sample (i.e. 2^{23}) is not divisible by U , there cannot exist a mapping between the uniform floating-point samples and uniform integers in $[0, U)$. However, for $U \ll 2^{23}$, this naive method can still be used as the bias introduced is mostly negligible.

A very simple solution to the problem of unbiased uniform integer sampling is rejection sampling, where we draw a pseudo random number and if the number is above or equal to the upper bound U , we reject and try again. The downside of rejection sampling in this situation is that it requires the generation of more pseudo random numbers, which, depending on the PRNG, might be slow. An alternative method is to use a hierarchical sampling strategy. As mentioned previously, if $U \ll 2^{23}$, the bias of using floating point numbers is mostly negligible. We can exploit this property by first uniformly selecting a range of integers and then, in a second step, drawing a uniform sample from this subrange. If both sampling steps now sample from a range which is significantly lower than U , the resulting sample is mostly unbiased.

2.4.3.1 Fast Dice Roller

To generate completely unbiased uniform integers within $[0, U)$, Lumbroso proposes the **fast-dice-roller** algorithm, which sequentially consumes single pseudo random bits (dice rolls), while retaining a uniform integer within $[0, v)$. The algorithm is structured such that v converges to U , after a potentially infinite amount of iterations. The **fast-dice-roller** algorithm is quite efficient in practice, needing only a slightly greater number of pseudo-random bits than the information-theory optimum.

3. Related Work

3.1 Inverse transform sampling

A straightforward sampling method is inverse transform sampling (ITS), which inverts the cumulative distribution function (CDF) to generate samples. Given a uniform random variable U and a CDF F , the random variable $X = F^{-1}(U)$, follows the same distribution as F . For discrete distributions, F can be represented by a prefix sum over the weights, where binary search can be used to evaluate $F^{-1}(u)$ in $\mathcal{O}(\log n)$ time.

3.2 Cutpoint method

An optimization of inverse transform sampling is the **cutpoint method** [CA74]. This method constructs a guiding table using a procedure similar to the inverse transform method itself. Specifically, it involves performing a binary search to locate G equally spaced values in $[0, \sigma)$, prior to the actual sampling. For each sample, the method uniformly selects an entry from the guide table, which provides an offset within F^{-1} . This offset then serves as the starting point for a binary search during the sampling phase. The number of entries in the guide table, G , is a tunable parameter: increasing G raises the initial construction cost but reduces the time required for sampling. Consequently, G allows adjustment of the balance between construction time and sampling efficiency. Figure 3.1 illustrates an example of the cutpoint method.

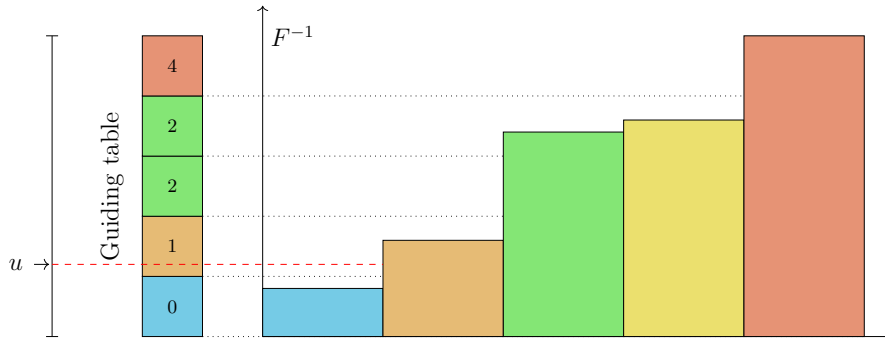


Figure 3.1: Illustrates the inverse transform sampling with a guiding table. The guiding table is constructed by initial binary searches, which are visualized with dotted lines. Sampling then involves drawing a uniform sample u . Here u maps to the second entry of the guiding table. Because this entry maps to element 1 and the next entry in the guiding table maps to entry 2, we know that the sample is in the range $[1, 2]$. The following binary search therefor only has to scan that range.

3.3 Alias Table

The Alias table [Wal77] is a data structure designed to efficiently sample from a discrete probability distribution with $\mathcal{O}(1)$ sampling time and $\mathcal{O}(n)$ space complexity. Formally, each entry in the table is a tuple (i, p, a) , where i is the index, p is the probability to not redirect and a is the entry in the table to which it is redirected. The entries in the alias table are chosen so that, when uniformly selecting a table entry i and redirecting to a with probability $1 - p$, each item is chosen with a probability proportional to its weight $\mathbb{P}[X = i] = \omega_i/\sigma$. Formally this is expressed in the invariant of an alias table:

$$\mathbb{P}[X = i] = \frac{1}{n}p_i + \sum_{j \in \{h | a_h = i\}} \frac{1}{n}(1 - p_j)$$

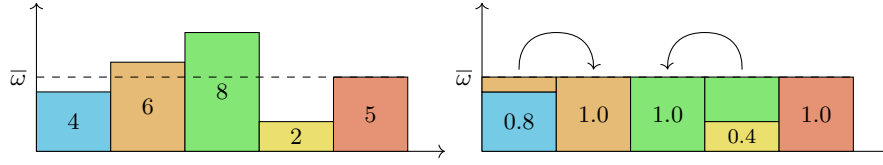


Figure 3.2: **left:** set of weights, with average weight $\bar{\omega} = 5$; **right:** illustrates a constructed alias table, given the weights, notably we see that all entries associated to weights below $\bar{\omega}$ redirect to entries associated with weights above $\bar{\omega}$.

3.3.1 Sequential Construction

A sequential approach to alias table construction [Wal77] iteratively redirects probability from over-sampled (heavy) weights to under-sampled (light) weights. With an initial sort, this method achieves a time complexity of $\mathcal{O}(n \log n)$. Later, a linear-time construction was proposed by Vose, maintaining two partitions: **heavy weights**, above the average, and **light weights**, below the average. This version follows a similar redirection process but requires only the maintenance of partitions, rather than repeatedly identifying minimum and maximum weights (Algorithm 1 shows a reference implementation). A further optimization, mentioned by Vose, allows for the lazy evaluation of partitions, reducing the space complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$; [HS22] provides a pseudo code implementation, which maintains two indices that lazily scan the weights for heavy or light weights.

Algorithm 1 Sequential construction algorithm by Vose

```

1: function SEQUENTIAL-CONSTRUCTION( $\Omega = \langle w_0, \dots, w_{n-1} \rangle$ )
2:    $\bar{\omega} = \frac{1}{n} \sum_{\omega \in \Omega} \omega$ 
3:    $h = \langle i \in 0..(n-1) : \omega_i > \bar{\omega} \rangle$  ▷ Heavy partition indices
4:    $l = \langle i \in 0..(n-1) : \omega_i \leq \bar{\omega} \rangle$  ▷ Light partition indices
5:    $i = 0, j = 0, w = w_{h[0]}$  ▷ Current light and heavy elements
6:   while  $i < |l| \wedge j < |h|$  ▷ Pack heavy element
7:     if  $w < \bar{\omega}$  then
8:       aliasTable[h[j]] = { $p = w/\bar{\omega}, a = h[j+1]$ }
9:        $w = (w + w_{h[j+1]}) - \bar{\omega}$ 
10:       $j = j + 1$ 
11:     else ▷ Pack light element
12:       aliasTable[l[i]] = { $p = \omega_{l[i]}/\bar{\omega}, a = h[j]$ }
13:        $w = (w + \omega_{l[i]}) - \bar{\omega}$ 
14:        $i = i + 1$ 
15:     end if
16:   end while
17: end function

```

3.3.2 Parallel Construction

The parallel splitting based alias table construction (PSA) algorithm [HS22] leverages a divide-and-conquer approach to efficiently construct alias tables on parallel architectures, particularly suited for GPUs [LHS21].

Theory: Assuming the set of weights $\Omega = \{\omega_1, \dots, \omega_n\}$ could be split into K distinct subsets $\Omega_1, \dots, \Omega_K$ such that each subset Ω_k has the same average weight as the overall set (i.e. $n_k \bar{\omega} = \sigma_k$, where $\sigma_k = \sum_{\omega \in \Omega_k} \omega$ and $n_k = |\Omega_k|$), each subset could be processed independently. Intuitively $n_k \bar{\omega}$ represents the sampled weights, because alias table entries are sampled uniformly (i.e. with weight $\bar{\omega}$) and σ_k represents the actual weight, with which a subset should be sampled. If we can process the K subsets independently, it is trivial to parallelize execution across K threads, without any communication. Although this exact partitioning is generally not feasible, this assumption highlights the general idea of the splitting-based construction.

To manage cases where equal partitioning is not possible, PSA relaxes the balance requirement by only requiring that $n_k \bar{\omega} \geq \sigma_k$ for each subset Ω_k and that each subsequent subset Ω_{k+1} contains an element with a weight large enough to cover any remaining unassigned weight from Ω_k . Because the sampled weight $n_k \bar{\omega}$ is greater than the required weight σ_k , we can still independently construct an alias table for each subset, by allowing Ω_k to "spill" its remaining weight to the next subset Ω_{k+1} . Specifically, if a subset Ω_k has an excess remaining weight, it redirects to the next subset to ensure that Ω_k is not over sampled. We call this relaxed requirement the partial balancing condition:

$$\forall k : n_k \bar{\omega} \geq \sigma_k \wedge \exists \omega \in \Omega_{k+1} : \sigma_k + \omega > n_k \bar{\omega}$$

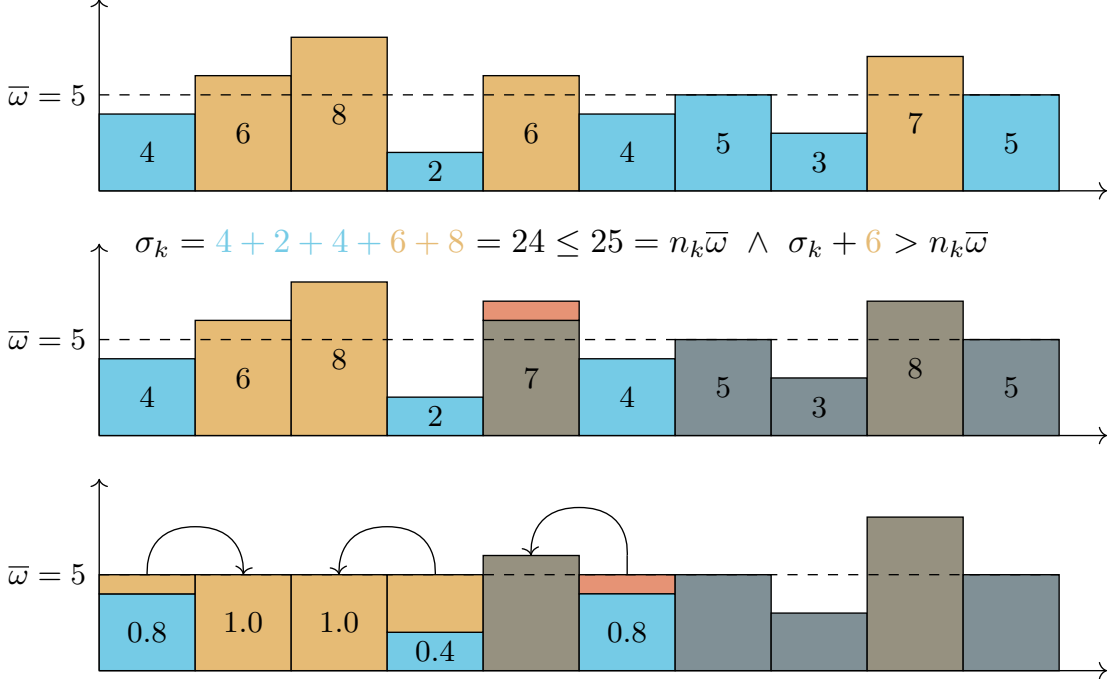


Figure 3.3: top: Set of weights with average weight $\bar{\omega} = 5$; **middle:** shows the splitting step with $n_k = 5$. Here splitting involves searching the interval $j \in [0, n_k]$, which finds the subproblem with $j = 3, i = 2$ and $\sigma_k = 24$. **bottom:** Shows the constructed alias table for the subproblem determined during the splitting step. Red regions indicate weight, which is "spilled" to the next subproblem.

Algorithm: PSA initially determines subsets Ω'_k with a fixed amount of weights $n_k = |\Omega'_k|$. The weight of each subset σ_k can be expressed with a prefix sum over the light and heavy elements:

$$\sigma_k(i, j) = \sum_{k < i} \omega_{l[k]} + \sum_{k < j} \omega_{h[k]}$$

where j is the amount of heavy elements within the subset, i is the amount of light elements (which is implicitly defined by $i = n_k - j$) and the arrays l and h contain the indices of light and heavy elements, respectively.

PSA selects such a subset with a binary search over $\sigma_k(i, j)$, to find the smallest j for which $\sigma_k(i, j) > n_k \bar{\omega}$. Because $\sigma_k(n - j, j)$ is monotone over j , we know that after the search terminates $\sigma_k(i + 1, j - 1) \leq n_k \bar{\omega}$ and $\sigma_k(i + 1, j) > n_k \bar{\omega}$, which implies that the subset $\Omega'_k = \{w_{l[k]} | k < i + 1\} \cap \{w_{h[k]} | k < j - 1\}$ fulfills the partial balancing condition. We refer to the process of determining Ω'_k as **splitting**.

The complete PSA algorithm involves determining K subsets Ω'_k , with $n_k = \lceil nk/K \rceil$. We define the amount of light and heavy elements in each subset i_k and j_k , furthermore, we define the spill $s_k = \sigma_k(i, j) - n_k \bar{\omega}$, as the remaining weight of the heavy element j_k . Two adjacent splits now define a subproblem Ω_k , with heavy elements $[j_{k-1}, j_k]$ and light elements $[i_{k-1}, i_k]$. To account for the "spillover" from previous subproblems each subproblem assumes its first heavy weight to be s_{k-1} . Further because we chose Ω'_k , such that it fulfills the partial balancing condition, each subproblem can redirect excess weight to the next subproblem, without communication.

Once the weights are split into K subproblems, PSA follows the sequential method for each subproblem. We call this step **packing**. Algorithm 2 shows a reference implementation of PSA.

Alternative Derivation: We shortly introduce an alternative derivation of the **splitting** step, which is equally correct. If we examine the sequential algorithm by Vose (see Algorithm 1), we find that it performs multiple iterations while only modifying the state variables: i the current light index, j the current heavy index and ω the remaining weight of the heavy element. In each iteration, we either pack the light or heavy element. If $w \geq \bar{\omega}$ we increment i and decrease ω by $\bar{\omega} - \omega_{l[i]}$. Similarly, if $w < \bar{\omega}$ we pack a heavy element, we increment j and increase ω by $\omega_{h[j+1]} - \bar{\omega}$. This pattern is fairly simple, as after any amount of iterations, i is the amount of packed light elements, j is the amount of packed heavy elements and ω is the remaining weight of the current heavy element, which is implied by i and j .

$$\begin{aligned} \omega(i, j) &= \omega_{h[0]} + \sum_{k=1}^j (\omega_{h[k]} - \bar{\omega}) + \sum_{k=0}^{i-1} (\omega_{l[k]} - \bar{\omega}) \\ &= \sigma(i, j) - (i + j - 1) \bar{\omega} \end{aligned}$$

Because Vose's implementation does not reorder the light or heavy elements and either increases ω if $\omega < \bar{\omega}$ or decreases ω if $\omega \geq \bar{\omega}$, it implicitly minimizes $\omega - \bar{\omega}$. Further, because $\sigma(n_k - j, j)$ is monotone over j , $\omega(n_k - j, j) - \bar{\omega}$ is also monotone, which means that a binary search (i.e. **splitting**) can be used to find the state of the sequential algorithm after n_k iterations. As expected, the binary searches of both derivations are identical:

$$\begin{aligned} \omega(i, j) > \bar{\omega} &\Leftrightarrow (\sigma(i, j) - (i + j - 1) \bar{\omega}) > \bar{\omega} \\ &\Leftrightarrow \sigma(i, j) > (i + j) \bar{\omega} \\ &\Leftrightarrow \sigma(i, j) > n_k \bar{\omega} \end{aligned}$$

PSA then involves starting the sequential algorithm with K threads, which begin execution after $n_k = \lceil kn/K \rceil$ elements were packed and only perform k/K iterations.

Algorithm 2 Pseudo code of the parallel splitting based alias table construction. All arrays or indices are zero indexed, to allow for easy translation into C like languages.

```

1: function PSA( $\Omega = \langle w_0, \dots, w_{n-1} \rangle$ )
2:    $\bar{\omega} = \frac{1}{n} \sum_{\omega \in \Omega} \omega$ 
3:    $h = \langle i \in 0..(n-1) : \omega_i > \bar{\omega} \rangle$  ▷ Heavy partition indices
4:    $l = \langle i \in 0..(n-1) : \omega_i \leq \bar{\omega} \rangle$  ▷ Light partition indices
5:    $\bar{h}_j = \sum_{k=0}^{j-1} \omega_{h[k]}$  ▷ Sum of the first  $j$  heavy weights
6:    $\bar{l}_i = \sum_{k=0}^{i-1} \omega_{l[k]}$  ▷ Sum of the first  $i$  light weights
7:   for  $k \in [1, K)$  dopar
8:      $(i_k, j_k, s_k) = \text{SPLIT}(\lceil nk/K \rceil)$ 
9:   end for
10:   $(i_0, j_0, s_0) = (0, 0, 0)$ 
11:   $(i_K, j_K) = (|l|, |h|)$ 
12:  for  $k \in [1, K]$  dopar
13:     $\text{PACK}(i_{k-1}, i_k, j_{k-1}, j_k, s_{k-1})$  ▷ Compute  $K$  splits.
14:  end for
15: end function
16: function SPLIT( $n_k$ )
17:   $a = 0$  ▷ Lower inclusive bound on the amount of heavy elements
18:   $b = \min\{n_k, |h|\}$  ▷ Upper inclusive bound on the amount of heavy elements
19:  while  $a < b$  do ▷ Invariant:  $\forall j < a: \sigma_k(n_k - j, j) \leq n_k \bar{\omega}$ 
20:     $j = (a + b) / 2$  ▷ Amount of heavy elements
21:     $i = \min\{n - j, |l|\}$  ▷ Amount of light elements
22:     $\sigma = \bar{l}_i + \bar{h}_j$  ▷ Use precomputed prefix sums
23:    if  $\sigma > n_k \bar{\omega}$  then
24:       $b = j$ 
25:    else
26:       $a = j + 1$ 
27:    end if
28:  end while
29:   $j = a - 1$ 
30:   $i = \min\{n - j, |l|\}$ 
31:  return  $(i, j, (\bar{l}[i] + \bar{h}[j + 1]) - n_k \bar{\omega})$ 
32: end function
33: function PACK( $\underline{i}, \bar{i}, \underline{j}, \bar{j}, s$ )
34:   $i = \underline{i}$  ▷ Current light element
35:   $j = \underline{j}$  ▷ Current heavy element
36:   $\omega = s$  ▷ Weight of the current heavy element
37:  while  $i < \bar{i} \wedge j < \bar{j}$  do
38:    if  $\omega < \bar{\omega}$  then ▷ Pack heavy element
39:       $\text{aliasTable}[h[j]].p = \omega / \bar{\omega}$ 
40:       $\text{aliasTable}[h[j]].a = h[j + 1]$ 
41:       $\omega = (\omega + \omega_{h[j+1]}) - \bar{\omega}$ 
42:       $j = j + 1$ 
43:    else ▷ Pack light element
44:       $\text{aliasTable}[l[i]].p = \omega_{l[i]} / \bar{\omega}$ 
45:       $\text{aliasTable}[l[i]].a = h[j]$ 
46:       $\omega = (\omega + \omega_{l[i]}) - \bar{\omega}$ 
47:       $i = i + 1$ 
48:    end if
49:  end while
50: end function

```

3.4 Scan

The scan operation computes the prefix sum for each element in a sequence and serves as a fundamental building block in parallel algorithms. For example, in parallel partitioning, an exclusive prefix sum over a predicate can determine write offsets for efficiently reorganizing data [HS05].

Parallel scan algorithms have been studied extensively, with early research originating in hardware circuit design, particularly in carry-look-ahead adders [Skl60; KS73; BK82]. A key consideration in circuit design is minimizing the latency. At a high level, we can imagine a hardware circuit as a network. In this view, the latency is related to the longest path from an input to an output, or in other terms, the depth of the network. When looking at parallel algorithms on GPUs, we can borrow ideas found in hardware design when optimizing for the latency of a kernel. Instead of minimizing the total amount of operations, we aim to minimize the depth of the network or iterations. The Kogge-Stone-Adder (KSA) is a hardware design that follows this principle; although it requires significantly more space on a die, it shows better performance as its network depth is smaller. Later, this method was generalized by Hillis and Steele and became a well-known parallel prefix scan algorithm.

In addition to minimizing the amount of iterations or the depth of the network, parallel algorithm implementations must efficiently utilize the properties of the underlying hardware, which means reducing the amount of communication over shared memory and, more importantly, the amount of barrier instructions.

In the following, we want to give a brief overview of methods found in open-source libraries (e.g. `cub` and `thrust`) to efficiently compute block-wise prefix scans (i.e. within a single workgroup).

3.4.1 Register scan

A very common optimization found in many compute shaders is to process multiple elements per invocation. In the context of a parallel scan, a single invocation loads multiple values from global memory and performs a sequential scan and publishes the aggregate. Afterwards, some mechanism determines the exclusive prefix of each invocation, which is then combined with the previous result to form a complete prefix sum.

3.4.2 Subgroup scan

Within a subgroup, invocations can communicate efficiently using subgroup intrinsics, eliminating the need for shared memory or execution barriers. Since all invocations within a subgroup execute in lockstep, intermediate values can be stored in registers, reducing latency compared to shared-memory-based approaches. We find that `cub` and `thrust` libraries implement a subgroup scan proposed by Sengupta *et al.* It is a variation of the parallel scan algorithm by Hillis and Steele that eliminates both workgroup execution barriers and shared memory by storing all intermediate values directly in registers and communicating with subgroup shuffle operations. Although this approach requires $\mathcal{O}(\log n)$ additional work compared to other work-efficient algorithms, it remains the most performant. This is because subgroup instructions are executed in a SIMT fashion, which means that there is no advantage in reducing the number of active invocations within a subgroup.

3.4.3 Ranked

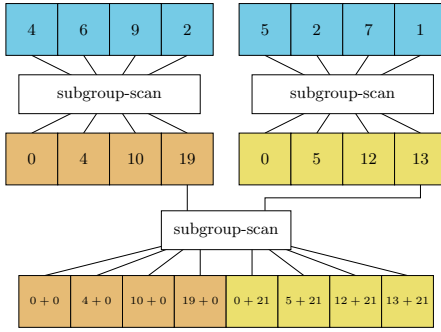


Figure 3.4: Ranked block scan

The ranked method computes a prefix sum across a workgroup. At a high level, the ranked approach is very similar to the parallel scan algorithm proposed by Hillis and Steele, with the major improvement being that values are moved between invocations, such that communication between subgroups is minimized. Figure 3.5 illustrates the general idea. We begin by computing a prefix sum for every subgroup. Afterwards, every subgroup publishes its aggregate to shared memory and a single subgroup performs an exclusive scan over the subgroup aggregates.

Lastly, all subgroups read their exclusive prefix from shared memory and add it to their local sum to compute a prefix sum over the complete block. Compared to a more naive implementation, this requires only two shared memory barriers and only requires communicating the exclusive prefix of every subgroup instead of every invocation. The `cub` and `thrust` libraries refer to this as a block-scan-warp-scan, we renamed this approach to **ranked** to avoid confusion with subgroup scans.

3.4.4 Raking

The `cub` and `thrust` libraries also provide an alternative method, which they call raking. Raking refers to a common shared memory pattern, where a single subgroup sweeps shared memory sequentially. With appropriate padding, this approach can completely avoid shared memory bank conflicts. After an initial register scan, the raking approach publishes all aggregates of the register scan to shared memory. Afterwards, it reduces the amount of parallelism to a single subgroup, where each invocation performs a sequential scan. The aggregates of the sequential scan are then combined with an exclusive subgroup scan. Afterwards the raking invocations publish their result back to shared memory and after a barrier instruction, all invocations read their exclusive prefix and combine it with the results of the register scan to form a block-wise scan.

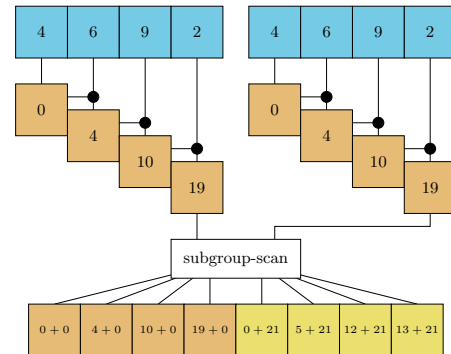


Figure 3.5: Raking block scan.

Compared to the ranked scans, NVIDIA claims higher overall throughput, as the ranking approach relies less on the inefficient subgroup scans. The `cub` and `thrust` documentation states that the **raking** method is likely the most performant method for large input sizes, where the **ranked** method is most applicable with few elements, because the average subgroup latency of the **ranked** approach is lower if the GPU is under occupied.

3.4.5 Arrays of arbitrary size

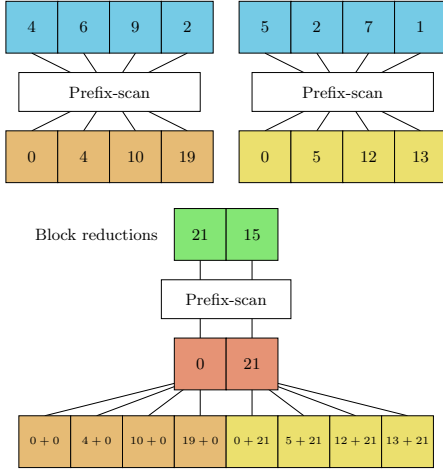


Figure 3.6: Block wise prefix scan with multiple dispatches

Scans of arbitrary sizes can be computed by independently building prefix sums over small blocks that can be held in shared memory [NVI; SHG]. The complete prefix scan can then be determined by performing an exclusive scan over the reductions of each block and then combining this exclusive prefix sum with the block-wise prefix sum (see Figure 4.1). If the prefix sum over the blocks cannot be held in shared memory, we apply this method recursively.

This requires multiple dispatches, which are synchronized over pipeline barriers. This may degrade performance as the GPU could become underutilized, especially if the number of elements becomes smaller in later iterations. Therefore, Sengupta *et al.* suggests adjusting the block sizes so that the complete prefix sum can always be computed in 2 iterations.

3.4.5.1 Single pass approach

Duane Merrill suggests an alternative method, which performs the complete scan in a single dispatch, where each element is read only once from global memory. Similarly to the previous method, it first computes a local aggregate over a block of the elements in shared memory. Afterwards it uses inter workgroup communication and synchronization to determine the exclusive prefix of its block.

For this, each workgroup now has a state.

- **X:** The workgroup has not yet computed its local aggregate.
- **A:** The workgroup has published its local aggregate to global memory.
- **P:** The workgroup has published its inclusive prefix to global memory.

Initially, all workgroups are in the **X** state. After determining the local aggregate, the state is changed to **A** and the aggregate is published to global memory. A workgroup can now determine its prefix, by initializing `exclusivePrefix = 0` and beginning the look back at the previous workgroup. Where if we read ...

- **X:** We spin wait until the state changes.
- **A:** Add the predecessors aggregate to the `exclusivePrefix` and move on to the preceding workgroup.
- **P:** Complete the look back, by adding the inclusive prefix of the predecessor to the `exclusivePrefix`.

Once a workgroup has determined its exclusive prefix it publishes its inclusive prefix and changes its state to **P**. Because of the occupancy bound execution on most GPUs, this lookback can be performed in constant time, as the worst-case depth is determined by the maximum occupancy.

Due to the inter workgroup synchronization required by this approach, a naive implementation requires the LOBE forward progress, as any workgroup requires workgroups with

lower IDs to be scheduled fairly; otherwise a workgroup might wait indefinitely. However, any algorithm which requires LOBE can still be implemented under OBE if we increment an atomic counter and use its results as the workgroup ID [Dua16].

On NVIDIA platforms the `single-dispatch parallel prefix scan` method approaches a memory throughput close to the memory bandwidth limits. More recent work [Lev21] has provided a portable Vulkan implementation, showing promising results, reaching a similar throughput on other platforms (e.g. AMD, Intel).

In addition to the possible performance improvements, the single dispatch nature of the method allows inlining this approach in other parallel algorithms, such as partitioning and stream compaction, allowing them to be performed in single dispatches as well [Dua16].

4. Parallel Scan

In this chapter, we present our parallel-scan implementations. Our implementations are built up from multiple components. An intra-subgroup scan, which operates within a single subgroup, a block scan, which uses the subgroup scan to compute a workgroup-wide prefix sum, and an inter-block scan mechanism, which computes a prefix sum of arbitrary size.

4.1 Subgroup scan

Our implementation relies on subgroup intrinsics provided by GLSL extensions, which are commonly available and have been promoted to a core feature of Vulkan. These intrinsics include dedicated functions, which compute inclusive and exclusive prefix sums over a subgroup. The underlying implementation of these intrinsics is highly driver-dependent and some platforms could even support dedicated hardware. We also provide an alternative implementation, which implements a Kogge-Stone style adder, which uses `subgroupShuffle` to efficiently communicate between invocations; this approach is similar to what we saw in the open source `cub` library.

4.2 Block Scan

Our implementation of a block scan is highly inspired by the open-source `cub` library, where we implement both `ranked` and `raking` methods. As block scan methods are mostly bound by the memory bandwidth, we want to look into the global memory access patterns in more depth.

Initially every invocation loads `ROWS` elements from global memory and stores them into a register float array of size `ROWS`.

```
1 void main() {
2     uint base = gl_GlobalInvocationID.x * ROWS;
3     float v[ROWS];
4     for (uint i = 0; i < ROWS; ++i) {
5         v[i] = elements[base + i];
6     }
7     ...
8 }
```

Listing 4.1: Global memory access pattern of the block scan methods

Although we ensure that the loop is fully unrolled, configurations where $ROWS$ is large may still produce uncoalesced memory accesses as memory is accessed with a large stride. Specifically, this occurs because on most architectures the largest memory access that can be issued in a single instruction is 16 bytes. For $ROWS = 8$ the compiler therefore has to produce two 16 byte transactions; this may lead to both memory accesses being packed into separate transactions, which potentially halves the memory throughput.

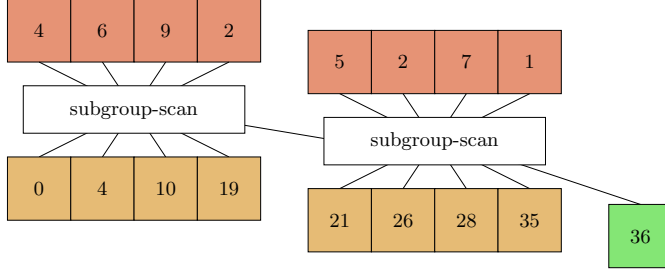


Figure 4.1: Ranked-strided block scan. Red represents global memory values, orange subgroup wide prefix sums and green the subgroup aggregate.

To mitigate this, we provide a modified version of the ranked approach, which uses a strided memory access pattern; we call this method **ranked-strided**. Instead of each invocation loading consecutive values from global memory, we read with a stride of the subgroup size. This way each SIMD read instruction results in a contiguous 128 byte transaction, which is the maximum transaction size on most GPUs. The downside

of this access pattern is that the register scans are no longer applicable; instead, we perform a subgroup scan for every row and combine the results with the subgroup broadcast instructions. In other words, instead of performing a sequential scan on the register level, we perform a sequential scan on a subgroup level. After the modified subgroup scan the **ranked-strided** method is equivalent to the **ranked** block scan.

Compared to the **ranked** and **ranking** block scans, the **ranked-strided** approach allows arbitrary selection of the $ROWS$ parameter, without any consideration of the underlying hardware. The downside of the **ranked-strided** approach is that it performs significantly more inefficient subgroup scans.

4.3 Inter block scan

Previously discussed methods allow the computation of prefix sums across a single workgroup or blocks. To combine the results of block-wise scans, we implement two different methods. The first method uses multiple dispatches, where the first dispatch computes a prefix sum per block as previously discussed; additionally, it publishes the aggregate of the block into a separate global memory buffer. Afterwards, a separate dispatch with a single workgroup computes an exclusive prefix sum over the aggregates. Lastly, another dispatch combines sum from the results of the exclusive prefix scan over the aggregates with the block-wise prefix sum from the first dispatch to form a complete prefix sum. We call this inter-block scan method **block-wise-scan**.

To perform the scan operation on arbitrary sizes, we could apply this method recursively; however, a much more performant solution is to adjust the parametrization of the block scans to the element count, such that a complete prefix can always be performed in 3 dispatches at most. For large N , this requires block scan methods with large block sizes. In section 4.2, we introduced **ranked-strided**, which allows larger block sizes; however, this is only possible to an extent as increasing the amount of elements per invocation requires more registers.

Another solution, which does not increase the register requirements, is to perform multiple block scans sequentially, while keeping track of the exclusive prefix of each block. We use this method when computing the exclusive scan over the block aggregates.

4.3.1 Single dispatch scan

A second method performs the complete scan within a single dispatch, using inter-workgroup synchronization with atomics. Our single dispatch implementation is closely aligned with previous work [Lev21]. In addition, we implemented all optimizations mentioned in the original paper [Dua16], including parallelization of the look-back.

4.3.1.1 Parallel look-back

Our implementation performs the look-back with a single subgroup. Although it is generally possible to parallelize the look-back across all invocations of the workgroup, it would involve synchronization over shared memory, which we expect to perform worse.

The original paper suggests reading the previous states and ...

- If all states are **A**, aggregate the results and move the look back window.
- If any state is **P** and there are no workgroups following this with state **X**. Aggregate all the following workgroups and terminate the look-back.

We implement this logic with very minor changes, where instead of checking if all states are **A**, we check if no state is **X**. Afterwards, we use `subgroupBallot` and `subgroupBallotFindLSB` to determine the step size.

4.3.1.2 Proving correctness

```

1 nonprivate buffer LookbackStates{
2     DecoupledState partitions[];
3 };
4 void publishAggregate(in uint partID, in float aggregate) {
5     partitions[partID].aggregate = aggregate;
6     atomicStore(
7         partitions[partID].state,
8         partID == 0 ? STATE_P : STATE_A,
9         gl_ScopeQueueFamily,
10        gl_StorageSemanticsBuffer,
11        gl_SemanticsRelease | gl_SemanticsMakeAvailable);
12 }
13 void publishPrefix(in uint partID, in float prefix) { ... }
14
15 void readState(in uint partID, out uint state,
16               out float aggregate, out float prefix) {
17     state = atomicLoad(
18         partitions[partID].state,
19         gl_ScopeQueueFamily,
20         gl_StorageSemanticsBuffer,
21         gl_SemanticsAcquire | gl_SemanticsMakeVisible);
22     if (state == STATE_A) {
23         aggregate = partitions[partID].aggregate;
24     } else if (state == STATE_P) {
25         prefix = partitions[partID].prefix;
26     }
27 }

```

Listing 4.2: Synchronization primitives

Availability and Visibility

Vulkan currently is the only graphics and compute API, which defines a memory model. Memory models provide guarantees for synchronization and describe how agents (invocations on the device) exchange data. Although in many ways the Vulkan Memory Model (VMM) inherits properties from the C++ memory model, it differs significantly in the context of availability and visibility of memory write operations. This is because GPU architectures often provide more fine grained control over data propagation, for example, an atomic operation on the device can define execution and memory scopes, to explicitly select at which point in the hierarchy the atomic operation should be resolved.

Listing 4.2 reduces the synchronization primitives of the decoupled look-back to two functions. Publishing the aggregate involves writing the aggregate to global memory and changing the state. Publishing the inclusive prefix sum has the same pattern, where we write to prefix instead of aggregate and modify the state to **P** instead of **A**. Other invocations later read this state and depending on the result load the aggregate or the prefix from global memory.

In the following, we want to show that our implementation, ensures that after reading the state **A**, we are guaranteed that previous writes to the aggregate are visible. As proving correctness with memory models, is rather verbose we omit trivial details such as race conditions, caused by multiple writes.

For the previous writes to the aggregate to be visible, we have to show that the relation **visible-to** exists between the write and read operations. If we ignore race conditions, the VMM states that a write operation is visible to the read operations if it is **location-ordered-before**.

For a write to be **location-ordered-before** a read, both have to access the same memory domain; in our case this is guaranteed by using `gl_ScopeQueueFamily` as the memory scope and both have to be **non-private**. In the Vulkan memory model and consequently SPIR-V, memory operations can be flagged as **non-private**, which means that they are intended for communication. In GLSL we can't explicitly control which memory writes are private or non-private; instead, we can qualify storage buffers with `nonprivate` to flag all reads and writes to these buffers as **non-private**.

The last condition of the **location-ordered-before** relation is more complex and states that the write operation has to be **program-ordered** after an **availability-operation**, which **happens-before** a **visibility-operation** **program-ordered** before the read (The VMM is more verbose here, and does not define the relations directly to the availability and visibility operations instead it defines it on the availability / visibility chains, in our context we can assume them to be the same as the operations themselves). The VMM states that an implicit **availability-operation** is inserted (directly) before an atomic operation with `MakeAvailable` and `Release` semantics. Further, an implicit **visibility-operation** is inserted (directly) after an atomic operation with `MakeVisible` and `Acquire` semantics.

In our case, the write to the aggregate is therefore **program-ordered** after the **availability-operation** and the read is **program-ordered** after the **visibility-operation**. The last relation, which we have to prove, is that the **availability-operation** **happens-before** the **visibility-operation** and because the atomic store **happens-after** the **availability-operation** and the atomic load **happens-before** the **visibility-operation**, we can instead show that the atomic load **happens-after** the atomic write (transitive).

To show that the atomic store **happens-after** the the atomic load, we have to show that **inter-thread-happens-before** relation exists between the atomic store and the atomic

load. Because both operations are atomic this is equivalent to showing that the atomic store operation with **Release** semantic **synchronizes-with** the atomic load operation with **Acquire** semantics.

Because both atomic operations operate on the same set of **memory-locations** and both define the same **memory-scope** (i.e. `gl_ScopeQueueFamily`), both are **mutually-ordered**, this together with the fact that the store performs a **Release** operation and the load performs a **Acquire** operation, means that the atomic store **synchronizes-with** with the atomic load, if the atomic load reads a value written by the atomic store.

This completes the proof, as the **synchronizes-with** relation between the atomic operations implies that **visibility-operation happens-after** the **availability-operation**, which means that the nonprivate write to aggregate is **visible-to** the nonprivate read of the aggregate, if the atomic load reads a value written by the atomic write (i.e. the state **A**).

To summarize; to ensure correctness, the VMM requires the aggregates and prefix sums to be placed in a storage buffer with the nonprivate modified, additionally writes of the aggregate have to be **program-ordered** after the atomic store operation, which changes the state to **A**. The atomic store operation is required to have the `gl_SemanticsMakeAvailable` flag set. Similarly, reading the aggregate is only valid **program-ordered** after an atomic load operation, which reads the state **A**. The atomic load operation is required to have the `gl_SemanticsMakeVisible` flag set.

Our single dispatch scan method implements all of these requirements, which shows correctness under the Vulkan memory model.

Forward progress

A potential downside of the single dispatch solution is that it requires the OBE forward progress guarantee as well as inter-workgroup synchronization over atomics. In this section, we want to formally prove that our implementation terminates under OBE.

To prove termination under OBE, we first want to prove termination under LOBE. As previously mentioned, LOBE states that a workgroup with a lower ID than a workgroup that has made a step is scheduled fairly. Because the look-back of a single workgroup is only dependent on workgroups with lower IDs, the look-back terminates under LOBE, as those workgroups are guaranteed to be scheduled fairly.

We now replace the workgroup ID with an atomic counter, which we increment at the start of each workgroup. Let the returned value of the atomic increment be X . We know that all workgroups with X lower than our X are scheduled fairly as they have made an initial step. We can therefore replace the workgroup ID with X and we guarantee that the algorithm will terminate under OBE.

4.4 Scan derivatives

The parallel scan primitive is one of the most important building blocks of many parallel algorithms. In the context of this work, we want to focus on parallel partitioning, while additionally computing the prefix sum over both partitions; we call this operation **scan-partition**. We provide multiple implementations, which utilize all parallel scan algorithms previously discussed, which include all block scan methods and all intra-block scan methods.

block-wise-scan-partition

The **block-wise-scan-partition** algorithm is separated into 3 compute shaders. The first dispatch determines the amount of heavy items and the reduction over the items and light items for each block. The next step performs 3 dispatches to compute the exclusive prefix sum over all 3 block aggregates (i.e. heavy count, heavy reduction, light reduction). The last step now combines the result of the previous step to determine a global exclusive prefix over the predicate, which determines the write offsets of global prefix sums over the weights. Compared to the **block-wise-scan** implementation, the **block-wise-scan-partition** uses a reduce-then-scan scheme to greatly reduce the amount of memory writes during the first step.

single-dispatch-scan-partition

We also provide a single-dispatch implementation, which performs the complete **scan-partition** operation within a single dispatch. We now calculate three distinct prefix sums rather than just one: the exclusive prefix sum over the predicate, the prefix sum over the heavy elements, and the prefix sum over the light elements. Because of the **single-dispatch** nature, all 3 scan operations can be performed within the same compute shader, where instead of computing one aggregate, we now compute 3. Similarly, during the decoupled look-back, we now determine the exclusive prefix of each block for each prefix sum. The last step is identical to the **block-wise-partition-scan**, where we use the exclusive prefix sum over the predicate to determine the write offsets for the prefix sums over light and heavy elements. The only major downside of inlining all 3 scan operations into a single compute shader is that the amount of registers required is potentially tripled, as we now have to store 3 partial prefix sums in registers. Following initial tests, we determine that to achieve maximum occupancy on our system, each invocation should utilize no more than 32 registers. If each invocation of the **single-dispatch-scan-partition** processes 8 elements, we do not surpass this limit, which means that this is not a performance-limiting factor.

This approach demands considerably fewer global memory accesses than the **block-wise-scan-partition** since elements are only retrieved from or written to global memory once.

4.5 Numerical stability

In this section, we want to address the numerical stability of our parallel scan implementations. All our sampling methods require the computation of a prefix sum over the weights, which are represented as positive floating-point values. Following the scan operation, all methods perform some variation of a binary search, which requires the prefix sums to be monotone for correct behavior. This guarantee is generally hard to achieve in parallel algorithms, as they reorder floating-point operations.

Subgroup intrinsics, such as the `subgroupInclusiveAdd`, do not explicitly provide any monotonicity guarantees; therefore, methods which utilize them to compute the local prefix sums rely on the driver implementation. However, we can assume that most platforms will implement those intrinsics with Kogge-Stone style adders, which do not provide any monotonicity guarantees.

Further, the single dispatch method reordered floating-point operations non-deterministically, as the depth of the look back is highly dependent on the current GPU state. We want to mention that the single dispatch method experiences significantly more instabilities in cases where the amount of workgroups is large as the exclusive prefix of a block shows similar numerical instabilities as the sequential method. Therefore we advise keeping the amount

of workgroups reasonably small to avoid numerical errors from accumulating during the look back.

Due to the numerical instabilities mentioned, the steps following the parallel scan cannot assume any monotonicity guarantees.

4.6 Uncompetitive methods

This section aims to briefly discuss methods that have been determined to be non-competitive based on preliminary tests. These methods are not included in the evaluation.

Work efficient block scans

We performed short experiments with work-efficient block scans, where we found them to be highly uncompetitive, as they do not account for the underlying SIMT execution model. Specifically, the work-efficient block scan performs two steps: down sweep and upsweep, where both require $\log n$ iterations compared to the more naive Kogge-Stone style adder, which only requires $\log n$ iterations in total. In practice, we found that work-efficient block scans are around 3.4 times slower than the block scans covered in section 4.2. We also attempted to replace only the subgroup scan with a work-efficient implementation, where we observed similar results.

Deterministic Single-Dispatch Scan

We can reorder the floating-point operations during the look back of the `single-dispatch-scan` by first performing the complete look-back without summing the aggregates and then once a block with state **P** is found, adding the aggregates directly to the inclusive prefix of this block. This method reverses the order of the floating-point operations during the look-back. This ensures that while determining the exclusive prefix of each block, the floating-point operations are deterministically ordered. Although this approach results in a fully deterministic prefix sum, it generally does not improve the numerical stability and only leads to an increase in the latency of around 10%.

Partition then scan

Instead of computing the `scan-partition` within a single dispatch, we could first compute a partition and afterwards two prefix sums over the partitions. Although we do not provide an exact implementation for this, preliminary experiments show that the latency of partitioning plus the latency of two parallel scan operations over half the amount of elements is around 10% slower than the `single-dispatch-scan-partition`.

5. Inverse transform sampling

Inverse transform sampling or inverse CDF method is one of the most commonly known sampling algorithms. We choose this method as a baseline to evaluate our alias table construction and sampling, because it requires very little construction involving only a parallel scan over the weights.

Inversion of the CDF involves a binary search. Our **baseline** implementation draws a uniform variable and performs the search independently with every invocation. The **baseline** approach does not involve any communication between invocations. This naive approach shows inefficient memory access patterns, as invocations within a subgroup follow divergent paths along the binary search tree.

We can improve the access patterns, by first drawing a uniform sample for a subgroup; then using the same uniform sample across invocations to reduce the search range to some upper limit U ; after the search range is smaller than U , every invocation draws another random sample and continues independently. We refer to this method as cooperative narrowing or just **coop**.

Alternatively, we provide an implementation, which replaces the initial cooperative narrowing of the search range, by using a p-array search. Although this does not reduce the total amount of memory accesses, it might reduce the amount of serial dependencies by lowering the amount of search iterations until the upper limit of U is reached.

The upper limit U is called the **section-size**. The **section-size** is a tuning parameter that should be selected based on the application requirements. Decreasing the **section-sizes** greatly increases the memory access patterns, which potentially increases the sampling throughput; however, it also increases the sampling variance, which might reduce the overall quality of a sample.

5.1 Cutpoint method

A very easy optimization of the inverse transform sampling method is the cutpoint method. It first constructs a guiding table, which is then used during sampling to replace the first couple of steps of the search. Our implementation uses a guiding table, where each entry covers a fixed amount of weight, or in other words we choose the size of the guiding table proportional to the amount of items. During sampling, each subgroup initially uniformly selects an entry in the guiding table and uses the indices stored in the guiding table as

a starting point for the search. Afterwards, each invocation of the subgroup draws an independent sample from this search range, using inverse transform sampling. In the context of the cutpoint method, we define the **section-size** as the average amount of items covered by each guiding table entry. Or formally:

$$G = \frac{N}{\text{section-size}}$$

where G is the amount of entries in the guiding table.

Similar to the **section-size** of inverse transform sampling with cooperative narrowing (**its-coop**), the **section-size** should be selected based on application requirements. Importantly, we want to note that the cutpoint method and **its-coop** can have different sampling variances for the same **section-size**, as both narrow the search range differently. **its-coop** only follows the same search path until the search range is smaller than the **section-size**, where the cutpoint method uniformly selects from distinct sections; these two mechanisms are not identical and it should therefore not be assumed that both achieve the same sampling variance, even if their **section-size** is the same.

5.1.1 Uniform sampling

We use the counter-based Philox algorithm to generate pseudo-random bit strings [SMDS11]. In the following, we want to briefly discuss how we map these random bit strings into uniform integer or floating-point samples.

When sampling with the **its-baseline** or **its-coop** method, we have to initially draw a uniform floating point number between $[0, \sigma)$. Our implementation first draws a uniform floating point number $\bar{u}$ between $[0, 1)$ (see 2.4.2) and multiplies it by the total weight σ . This method performs well and is mostly unbiased if $n < 10^6$. For larger distributions, the resolution of the $\bar{u}$ (i.e. 23bits of the mantissa) is too low, which leads to biased sampling. This problem is essentially unavoidable in our implementation¹, because our accuracy is bound by floating point precision and using double-precision numbers incurs too much overhead.

Sampling the cutpoint method involves two separate sampling steps, where we first select a uniform entry of the guiding table and then draw a uniform weight within the subrange described in the guiding table. A surprising side effect of this two-stage sampling process is that it often reduces the sampling bias for large distributions. When selecting the guiding table, we draw a uniformly distributed integer from the range $[0, G)$. We implement this by first drawing a pseudo-random floating-point number and multiplying it by G . As G is often significantly smaller than n (and $G \ll 2^{23}$), the bias introduced by using floating-point numbers is mostly negligible. For any distribution larger than 10^8 , we generally advise either increasing the **section-size** or using the **fast-dice-roller** algorithm to select the guiding table entry [Lum13].

¹The numerical instabilities of both the **scan** and sampling stages can be avoided by building block-wise scans, sampling then involves first selecting a block and then selecting a sample from within that block. We do not provide such a implementation, due to time constraints.

6. Alias method

6.1 Parallel splitting based alias table construction (PSA)

In this chapter, we want to discuss our different PSA implementations and optimizations. Before we delve into implementation details, we want to highlight the separate steps and serial dependencies between those steps. Before the **splitting** and **packing** step, PSA requires a precomputation of the partition indices over heavy and light weights, as well as a prefix sum over both the heavy and light weights. To evaluate the partition, we require the average weight; this is a strong serial dependency as we can't partition any weights without determining the average weight. Contrasting to this, the prefix scan over the light and heavy partitions only has a weak serial dependency to the partition step, as the parallel scan can begin after only some weights are partitioned. The following **splitting** step also only shows a weak serial dependency to the parallel scan as the computation of a single split with n_k weights only requires the prefix sums to be computed up to n_k elements. The **packing** step again shows a weak dependency as a single **packing** operation only depends on two splits to be done. This means that we can theoretically begin packing buckets while some splits are still being computed.

Any strong serial dependency is unbreakable no matter what synchronization we apply. However, weak serial dependencies can be broken if we apply more fine-grained synchronization. In general, such synchronization is hard to achieve as most computing frameworks only provide coarse-grained synchronization primitives such as pipeline barriers. However, we can break the dependencies by performing multiple steps within a single kernel, which allows resolving dependencies within a single compute shader.

The computation of the average weight is trivially reducible to the problem of parallel reduction. We found that performing a workgroup local reduction and accumulating the

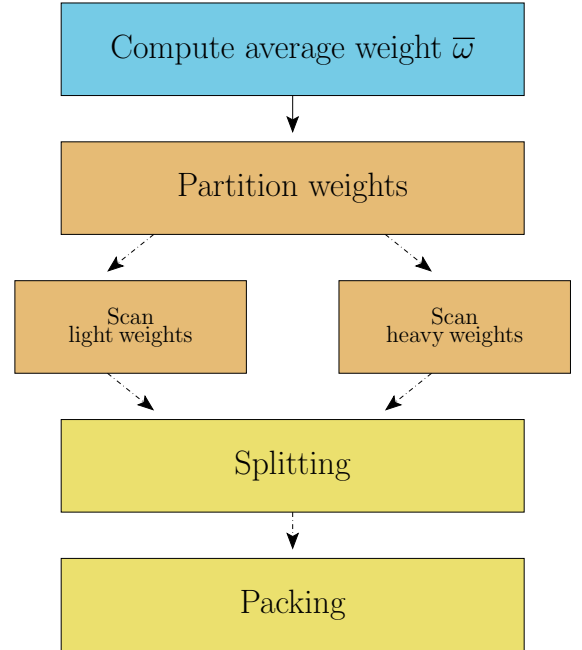


Figure 6.1: Illustrates all steps of the PSA algorithm including its serial dependencies. Solid lines represent strong serial dependencies, where dotted lines show weak dependencies.

result with a global floating point atomic performs best. Although floating point atomics are not available on all platforms, lots of alternatives such as tree reductions perform similarly. As the problem of reduction has been studied in depth, we will not cover implementation details here. Following the computation of the average weight, we use the **scan-partition** methods from section 4.4 to compute the partition indices and the prefix sums over both partitions.

6.1.1 Splitting

As the **splitting** step does not require any communication, our baseline implementation, follows the reference code in Algorithm 2, which implements a simple binary search per invocation. Compared to the binary search found in the inverse transform sampling, memory access patterns of this implementation are surprisingly efficient as invocations within a subgroup are initialized with similar n_k . As a result, invocations within a subgroup follow similar paths along the search tree.

We additionally implement a **splitting** method, which uses p-Array search instead of binary search. Similarly to **its-coop**, we confine the search range using a p-array search, which minimizes serial loop dependencies, while incurring a slight overhead due to the communication between invocations.

Numerical side effects

Implementations of the splitting step are generally straightforward. However, a key issue arises due to numerical instabilities in the preceding parallel scan (see section 4.5). Any non-monotonic sequence in the prefix sum over light and heavy elements may result in splits where (i, j) are not monotonically increasing. For example:

$$\begin{aligned} n_{k-1} &= 20 \xrightarrow{\text{split}} (11, 9, s_k - 1) \\ n_k &= 22 \xrightarrow{\text{split}} (10, 12, s_k) \\ n_{k+1} &= 24 \xrightarrow{\text{split}} (12, 12, s_{k+1}) \end{aligned}$$

These situations are especially likely to occur if the **split-size** = $\lceil n/K \rceil$ is small and the amount of elements is significantly larger than 10^6 . This creates easy-to-miss edge cases in the subsequent packing step, as it can create subproblems with varying size and $\underline{i} > \bar{i}$ or $\underline{j} > \bar{j}$. When we imagine the splitting step as finding the state of the sequential case after n_k steps (see 3.3.2), it's trivial to see that the constructed subproblems are invalid as splits with n_{k-1} and n_k would have packed elements in a different order, which means that the spill s_{k-1} does not describe the remaining weight properly.

In our implementation, these instabilities are unavoidable, however, we believe that using block-wise scan could potentially improve the numerical stability, as the binary search now first selects a block and then selects a number of heavy elements from within that block. This would avoid adding the block reductions to the block scans, which reduces the amount of floating-point rounding. Due to time constraints we do not provide such an implementation.

6.1.2 Packing

Similarly to the splitting step, packing does not involve any communication; therefore, our baseline implementation follows the reference code in Algorithm 2, where a single invocation performs one packing operation. The following gives an overview of the memory accesses performed during the packing step given the subproblem $(\underline{i}, \bar{i}, \underline{j}, \bar{j})$:

- The contiguous subrange $h[\underline{j} \dots \bar{j}]$ is access sequentially.
- The contiguous subrange $l[\underline{i} \dots \bar{i} - 1]$ is accesses sequentially.
- The subrange $\omega_{h[\underline{j}], \omega_{h[\underline{j}+1]}, \dots, \omega_{h[\bar{j}]}$ is accessed sequentially.
- The subrange $\omega_{l[\underline{i}], \omega_{l[\underline{i}+1]}, \dots, \omega_{l[\bar{i}]}$ is accessed sequentially.
- The subrange $\text{table}[h[\underline{j}]], \text{table}[h[\underline{j} + 1]], \dots, \text{table}[h[\bar{j}]]$ is written sequentially.
- The subrange $\text{table}[l[\underline{i}]], \text{table}[l[\underline{i} + 1]], \dots, \text{table}[l[\bar{i}]]$ is written sequentially.

We find that invocations within the same subgroup access similar memory locations, with a stride proportional to the subproblem size. If the subproblem size is sufficiently small, global memory accesses can be partially coalesced, as threads within a subgroup access the arrays in similar memory locations. However, as the subproblem size grows, this partial coalescing of memory transactions becomes more unlikely. With large subproblem sizes, this leads to every invocation issuing 3 uncoalesced memory transactions per iteration, which leads to significant pipeline throttling as the Load Store Unit (LSU) can't keep up.

A common approach to improve memory coalescing is to load values first into shared memory. Unfortunately, this approach requires too much shared memory, for large subproblem sizes. For example, assuming a subproblem size of 32, every invocation requires 384 bytes; assuming a single SIMT-processor can schedule 48 subgroups, this would require around 600KB of shared memory per SIMT-processor, which most consumer hardware does not support. Therefore, this approach would lead to a significant reduction in the maximum occupancy. This leads us to the conclusion that loading all values to shared memory is not an option.

We suggest an alternative approach, which performs a single pack operation, within one subgroup instead of one invocation. Although the packing is still performed by a single invocation, the memory transactions are performed cooperatively, which ensures that memory reads to the h and l arrays are coalesced. We store the values of the memory transactions directly in registers and access them from the invocation, which is currently performing the pack operation, via subgroup intrinsics (i.e. `subgroupShuffle`). Memory transactions of the weight and alias table arrays are not contiguous so we can't assume them to be performed within a single transaction. Nevertheless, this approach still reduces the amount of memory transactions for those arrays as we can assume the h and l arrays to contain monotonically increasing indices as most parallel partition algorithms are stable. At this point we also want to address the decision to store the prefetched values in registers instead of shared memory. An interesting property of most modern GPU architectures is that the size of the register file is often significantly larger than the shared memory. Further by leaving large parts of the shared memory unallocated, GPU architectures with a unified L1 and shared memory, implicitly increase the L1 cache capacity, which can help hide some of the uncoalesced memory accesses.

We expect that this subgroup-based approach improves the overall subgroup latency, however, it also requires significantly more subgroups to be executed in total. For small amounts of subproblems, where all subproblems can be processed concurrently, we expect this method to perform best as the overall latency is only bound by the latency of a single subgroup. However, we don't expect any significant improvements as the subgroup-based approach most likely does not reduce the subgroup latency by a factor greater than the subgroup size.

To allow more fine-grained control, we also provide another implementation, which is parameterized over the amount of invocations T , that cooperatively perform a single pack operation. As this implementation still relies on communication within a subgroup, T is bound by the subgroup size.

6.1.2.1 Uncompetitive Methods

Cyclic buffer

Instead of fetching heavy and light elements in chunks of T elements, we can implement a cyclic buffer, where we load new heavy / light elements immediately after one is consumed. Importantly, we make sure that the register to which we load is not touched until they are actually needed. The idea behind this approach is that instead of improving the memory coalescing, we hide the long latency memory accesses behind math and branching instructions. Implementations with a cyclic buffer generally perform around 50% worse.

6.1.3 Split-Packing

As mentioned in the beginning of this chapter, the packing step only has a weak serial dependency on the splitting step. We can exploit this property by combining the splitting and packing kernels into a single compute shader. One minor advantage of this approach is that we avoid a pipeline barrier, which could improve the overall utilization of the GPU. Furthermore, by combining both steps, we avoid writing the results of the splitting step to global memory, which for parametrization with lots of subproblems (i.e., lots of splits) reduces the total amount of global memory operations significantly.

To resolve the dependency between the packing and splitting steps, we provide two different methods depending on the amount of invocations T that cooperatively pack a subproblem.

Single invocation per subproblem

Every invocation independently computes the split $(i_{\text{gid}}, j_{\text{gid}}, \delta_{\text{gid}})$. Afterwards, each invocation performs the packing step for the subproblem $(i_{\text{gid}}, i_{\text{gid}+1}, j_{\text{gid}+1}, j_{\text{gid}}, \delta_{\text{gid}})$. The values of i_{gid} , j_{gid} and δ_{gid} are already available in local registers, the values $i_{\text{gid}+1}$ and $j_{\text{gid}+1}$ are available in neighboring invocations. If the neighboring invocation is within the same subgroup we can use `subgroupShuffleDown` to exchange values; otherwise, we communicate those values over shared memory. Notably, this method requires very little additional work, where only the last invocation of a workgroup performs a redundant split operation.

Multiple invocations per subproblem

If we allocate more than 1 invocation per subproblem, we can avoid communication over shared memory completely, by performing additional work. We split the invocations assigned to each subproblem into two groups, where the first group computes the split $(i_{\text{gid}}, j_{\text{gid}}, \delta_{\text{gid}})$ and the second group computes $(i_{\text{gid}+1}, j_{\text{gid}+1})$. Afterwards, all invocations cooperatively work on the subproblem $(i_{\text{gid}}, i_{\text{gid}+1}, j_{\text{gid}+1}, j_{\text{gid}}, \delta_{\text{gid}})$. This method is significantly less work efficient, as it potentially computes the same split with multiple invocations. However, in practice, we expect that these inefficiencies are hidden by the SIMT processors' local L1 cache.

6.2 Sampling

We build alias-tables mainly because they allow us to sample in constant time. Algorithm 3 shows pseudo-code, which draws a single sample from an alias table.

Our baseline implementation follows the reference implementation, where a single invocation performs this operation independently. Similar to the optimizations we introduce for inverse transform sampling, we can improve the memory access patterns by first selecting a section of the alias table within a subgroup and then drawing samples from this section. We call this sampling strategy `sectioned-sampling`. The `section-size` is a tuning

Algorithm 3 Sampling from an alias table in constant time.

```

1: function SAMPLE-ALIAS-TABLE(aliasTable =  $\langle(p_0, a_0), \dots, (p_{n-1}, a_{n-1})\rangle$ )
2:    $u_1 \sim \text{Uniform}_{\mathbb{N}}(0, n - 1)$   $\triangleright$  Draw uniform integer between 0 and  $n-1$ .
3:    $u_2 \sim \text{Uniform}_{\mathbb{R}}(0, 1)$   $\triangleright$  Draw uniform number between 0 and 1.
4:   if  $p_{u_1} < u_2$  then
5:     return  $u_1$ 
6:   else
7:     return  $a_{u_1}$ 
8:   end if
9: end function

```

parameter that should be selected based on the application requirements, where smaller section sizes increase the sampling variance, while improving the memory access pattern. We again want to note that different sampling algorithms with the same **section-size** may show significantly different sampling variances.

6.2.1 Uniformly select an entry

In this section, we briefly emphasize key considerations necessary for uniformly selecting an alias-table entry. Our implementation uses the counter-based Philox algorithm to generate pseudo-random bit strings [SMDS11]. The first step in the sampling process involves uniformly selecting an alias table entry by drawing a uniformly distributed integer in $[0, n)$. The naive method described in section 2.4.2 uses a uniform floating-point number in $[0, 1)$ and multiplies it with n . For large distributions, this process is biased as the floating-point number simply has too few bits. To ensure an unbiased selection process, we use the **fast-dice-roller** algorithm to select an alias entry without bias [Lum13].

If every invocation draws an independent sample (i.e. as in **baseline** sampling method), the additional computation of the **fast-dice-roller** algorithm can be fully hidden behind long latency memory operations. As the **sectioned-sampling** method improves the memory access patterns, which potentially reduces the average latency of memory operations, hiding the **fast-dice-roller** algorithm behind memory operations is no longer feasible. Nonetheless, since the **sectioned-sampling** technique, akin to the cutpoint approach, initially picks a uniform integer from a narrower range, we can generate uniform integer samples using floating-point numbers without notable bias. This means that, our implementation of **section-sampling** isn't completely unbiased, but due to numerical instabilities during construction having a larger effect on bias, we tolerate a minor bias while sampling.

For applications, which still want to use the alias-method and require unbiased samplers or work with distribution sizes larger than 10^8 , we generally advise using the **fast-dice-roller** algorithm or rejection sampling to select the alias entry uniformly.

7. Evaluation

In the subsequent chapter, we perform performance measurements to compare our implementations of weighted random sampling.

All evaluations are conducted using an NVIDIA GeForce RTX 4070 consumer graphics card, and Table 7.1 presents a summary of the specifications. Here we want to specifically highlight the size of the L2 cache. Compared to the previous Turing architecture by NVIDIA, all GPUs with the Ada Lovelace architecture feature significantly larger L2 caches. For example, the high-end RTX 3090 Ti only contains 6 MB of L2 cache, compared to the consumer model of the Ada Lovelace architecture, which features 36 MB, with the same clock speed. This trend is not isolated to NVIDIA architectures; on AMD’s most recent RDNA3 and RDNA4 architectures, we see a similar trend, where they implement a large L3-cache (Infinity Cache). Because of this large jump in hardware design, we try to address caching effects in our evaluation.

Name	NVIDIA GeForce RTX 4070
Architecture	Ada Lovelace (AD104)
Memory Size	12GB
Memory Bandwidth	504.2GB/s
Max memory clock speed	10501MHz
Max subgroups per SM	64
SM count	46
L2-cache	36MB
L1-cache (per SM)	128KB
Max shared memory per SM	100KB

Table 7.1: Specifications of the NVIDIA GeForce RTX 4070 graphics card.

We perform all measurements with Vulkan timestamp queries, over multiple iterations. To ensure consistent measurement results, we lock the memory clock frequency to 10501MHz. Preliminary benchmarks show that not locking the memory clock speeds leads to significant

artifacts in the latency measurements as the device switches between idle and performance modes, between measurements.

Importantly, we generate randomized inputs for each measurement iteration to avoid caching effects. If not stated explicitly, we generate uniformly distributed inputs between 0 and 1. Because some measurements require large input sets and generation of large input sets on the host is relatively slow, we generate all input data on the GPU. One major side effect of this is that a portion of the input elements might be present in L2, before we dispatch the compute shader. When comparing memory throughput this can result in measurements above the VRAM memory bandwidth limit. We still provide such measurements, where the complete input is present in the L2 cache, as it can give us insight into the performance of implementations, which would otherwise be limited by global memory latencies. To distinguish between these two measurement styles, we implement a small compute shader, which writes random values into a contiguous chunk of memory, which is larger than the L2. This method ensures that no input values are present in the L2 before dispatching compute shaders. In subsequent Figures, dotted lines depict cached measurements, while solid lines represent measurements where we either flushed the cache explicitly or the input size is too large to fit into the 36MB L2.

7.1 Memory bandwidth baseline

We initially evaluate the memory bandwidth limits of our device. As a reference, we implement a compute shader, which copies elements from a source to a target buffer. Figure 7.1 shows the memory throughput of this compute shader. In particular, we see that without flushing the L2, we can observe memory throughputs significantly above the VRAM bandwidth, as data does not have to be read from global memory. We use these measurements as a reference for compute shaders, which are limited by the memory bandwidth.

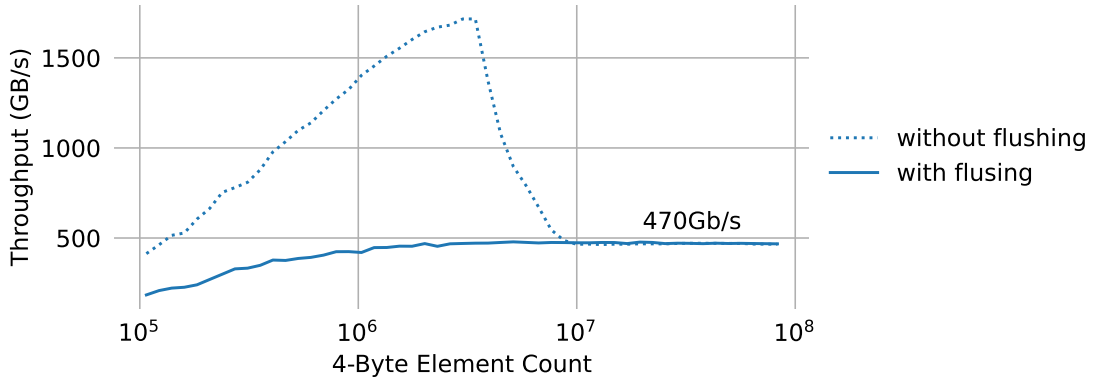


Figure 7.1: Observed memory bandwidth with and without a initial flush of the L2 cache.

7.2 Prefix Sum

Block-scan

Figure 7.2 shows the observed memory throughput of our **block-scan** methods. When flushing the L2 before each measurement, we don't observe any significant difference between all 3 block scan methods. However, without flushing the L2, we observe that the **ranked-strided** block scan consistently performs best.

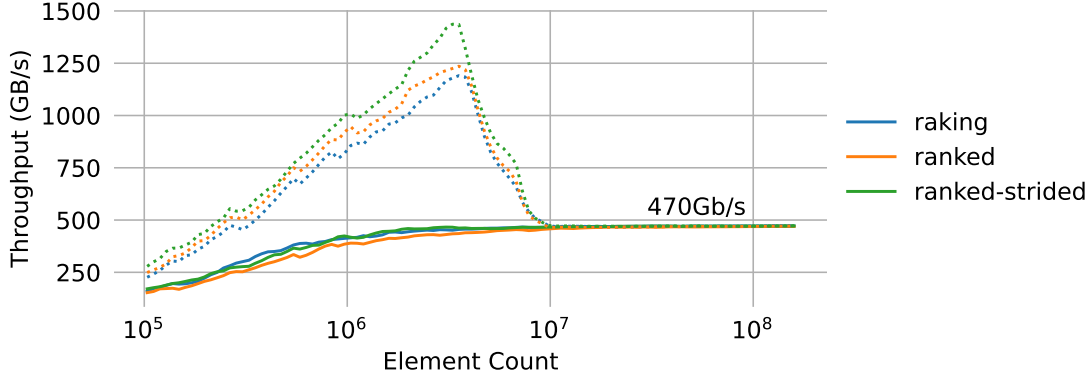


Figure 7.2: Observed memory throughput of block scan methods. Solid lines represent measurements, where we explicitly flushed the L2 cache.

As mentioned in section 4.1, we implemented a custom subgroup scan with a Kogge-Stone-Adder on a subgroup scope, which avoids execution and memory barriers. We don't find any significant difference compared to the `subgroupInclusiveAdd` instruction provided by the GLSL extension.

Inter-block scan

We cover inter-block scans in section 4.3. Figure 7.3 shows the observed throughputs of both the block-wise and the single dispatch methods. All inter-block scan methods internally use the best performing block scan method (i.e. `ranked-strided`). We provide measurements with and without flushing the L2.

We find that the `single-dispatch-scan` consistently performs best, reaching memory throughputs close to the optimal (i.e. 470GB/s). The `block-wise` scan method only reaches half of the possible throughput, which is expected as it requires double the amount of memory accesses. Here we want to shortly mention that our block-wise implementation is far from optimal, as it does not utilize a reduce-then-scan scheme, which would reduce the total amount of memory accesses. However, even with such optimizations, the total amount of memory accesses of a block-wise scan would still be significantly larger than the amount of memory accesses required for the `single-dispatch-scan`, we can therefore safely assume that the `single-dispatch-scan` will generally outperform `block-wise` implementations.

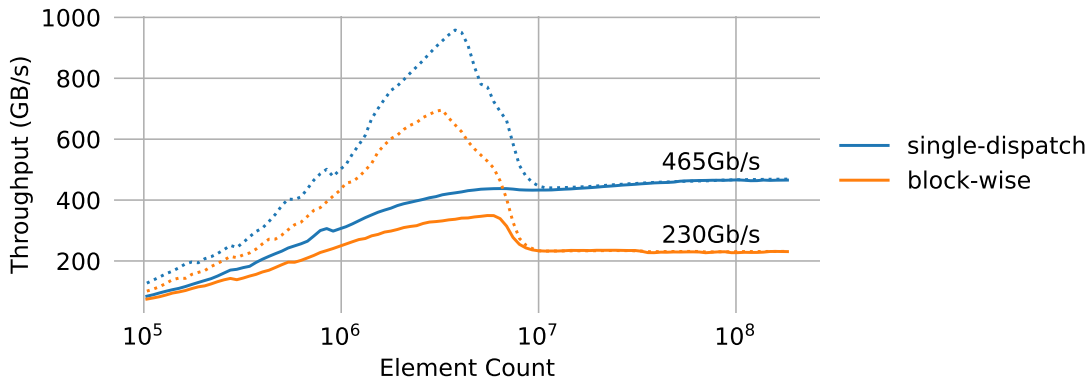


Figure 7.3: Memory throughputs of the inter block scan methods. Solid lines are measurements taken with a flushed L2 cache, and dotted lines are measured without flushing the L2.

7.2.1 Prefix over Partitions

In section 4.4 we briefly discuss the **scan-partition** operation. Figure 7.4, shows the observed memory throughputs of a kernel, which performs a partition and additionally computes the prefix sum over both partitions. We evaluate two methods; the first method is based on the **single-dispatch** scan, and the second method is based on the **block-wise** scan method. Both methods shown in Figure 7.4, write the partition indices as well as the prefix sum over both partitions. We find that the methods based on the **single-dispatch** scan perform significantly better than the methods based on the **block-wise** scan. Compared to the memory throughputs of scan methods (see Figure 7.3), we find that the additional global memory write operations only lead to a 86% reduction in the memory throughput.

Further, we performed measurements, where in addition to the partition indices we also write the partition elements back to global memory, this requires additional memory write operations. Figure 7.5 illustrates the element throughput, with and without writing the partition elements. We observe that writing the partition elements, in addition to the indices, reduces the latency by a factor of 0.8, which is expected as not writing the elements reduces the amount of global memory operations by exactly 80%.

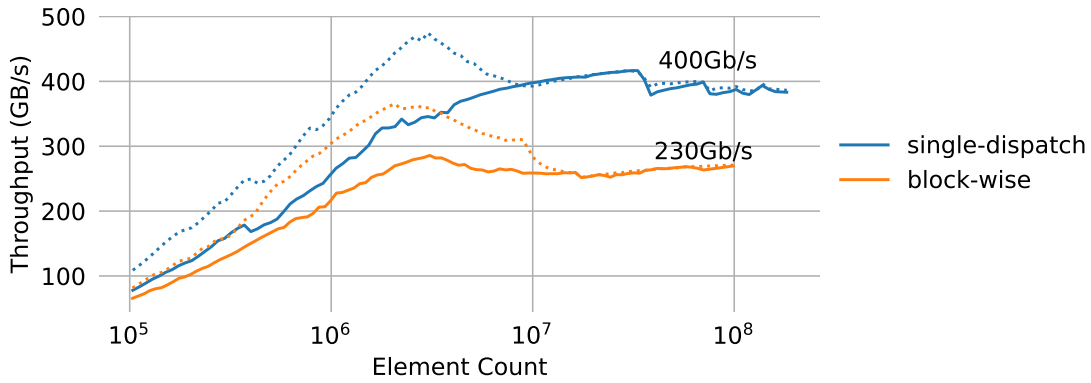


Figure 7.4: Memory throughput of our **scan-partition** methods. Solid lines show measurements taken with a flushed L2 cache, and dotted lines show measurements without flushing the L2.

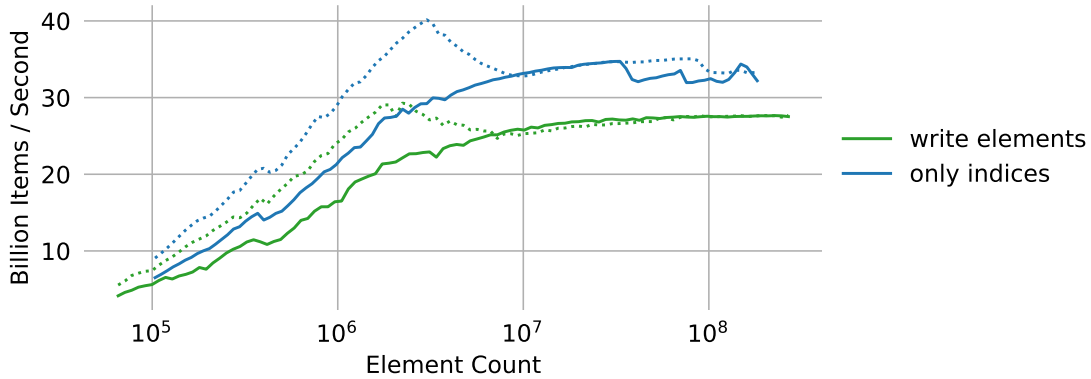


Figure 7.5: Comparison of the element throughput of the **single-dispatch-scan-partition**, with and without writing the partition elements back to global memory. Solid lines show measurements taken with a flushed L2 cache, and dotted lines show measurements without flushing the L2.

7.2.2 Numerical stability

We additionally conduct small tests to evaluate the numerical stability of our `single-dispatch-scan` implementation. As a reference, we use a sequential scan method, with Kahan summation and 16-byte floating point numbers (`long double`). Figure 7.6 shows the relative error (i.e. the absolute error divided by the sum of the elements) of the `single-dispatch` method. We additionally show the numerical stability of a sequential scan without Kahan summation and a sequential scan with Kahan summation using only single-precision floating point numbers.

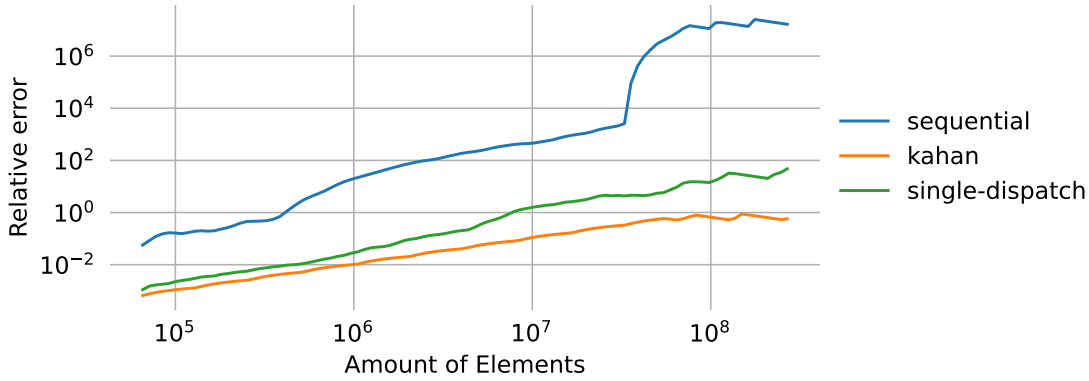


Figure 7.6: Relative error of the `single-dispatch-scan` compared to a Kahan summation with long doubles.

We find that although our `single-dispatch-scan` is significantly more stable than a naive sequential scan, it is still considerably worse than a sequential scan, which uses a Kahan’s summation algorithm.

7.3 Inverse Transform Sampling

We cover inverse transform sampling in chapter 5, as a baseline we evaluate the inverse transform sampling without cooperative narrowing, meaning that each invocation performs an independent search. We introduce an optimization, which cooperatively narrows the search range within a subgroup, which improves the memory access pattern significantly. In chapter 5 we cover two ways to implement the cooperative narrowing. The first method draws a single uniform sample for a subgroup and performs the identical binary search with all invocations of the subgroup. The second method replaces the binary search with a p-array search. We don’t observe any significant difference between both approaches. We also evaluate the cutpoint method, which replaces the cooperative narrowing with a guiding table.

Figure 7.7 shows the sampling throughput of the baseline, the cooperative narrowing and the cutpoint method, when drawing 10^7 samples. For the cooperative narrowing and the cutpoint method we select a `section-size` of 128 items, as it provides a good balance between sampling variance and throughput.

We find that inverse transform sampling, with cooperative narrowing significantly increases the performance, while only reducing the quality of the samples marginally. We additionally find that the cutpoint method consistently performs best; it is also the only method which efficiently utilizes the L2 cache, as it requires significantly less global memory accesses, which leads to the guiding table not being evicted from the cache.

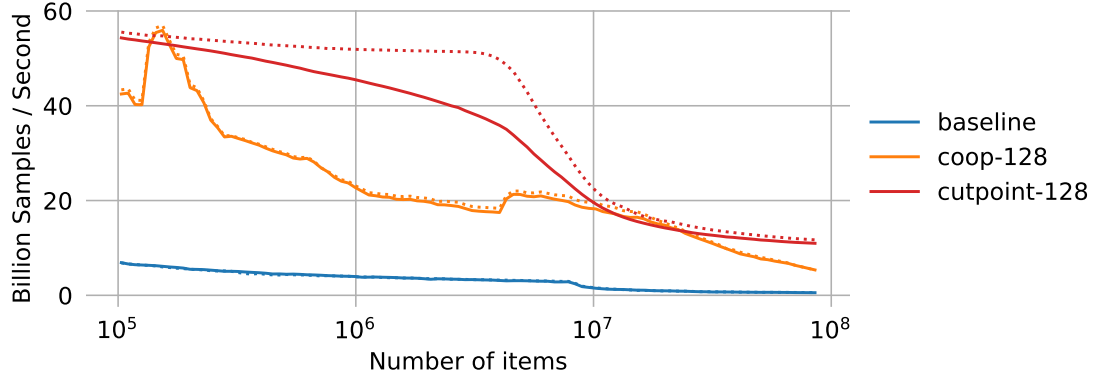


Figure 7.7: Sampling throughput of the inverse transform sampling methods. We measure the sampling throughput by drawing 10^7 samples, increasing the amount of samples further does not increase the sampling throughput.

In many applications, we find a pattern where a set of weights is only used once for sampling. In Figure 7.8 we show the latency of the cutpoint method, including the construction time and sampling of 10^7 samples. Importantly, we don't flush the L2 cache between the construction and sampling and instead only synchronize with a pipeline barrier. We observe that when the size of the distribution is considerably smaller than the number of samples collected, the extra effort required to construct the guiding table is almost insignificant.

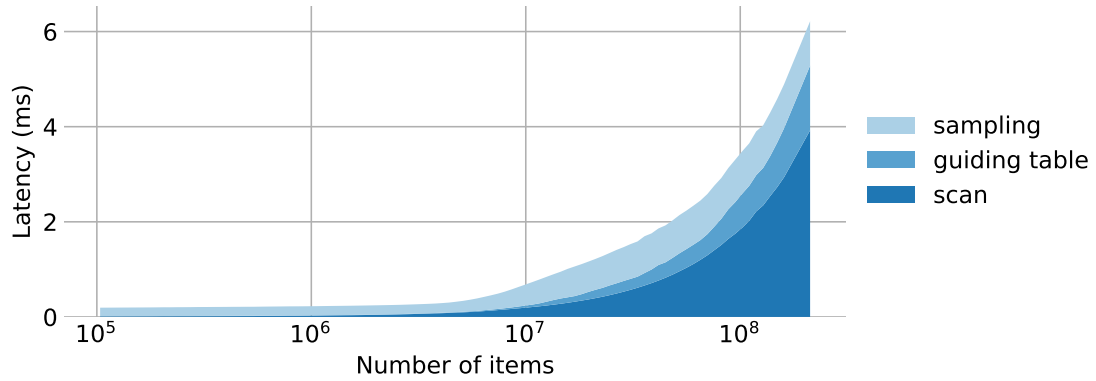


Figure 7.8: Shows the latencies of the individual steps of the cutpoint method, including construction and sampling of 10^7 samples.

7.3.1 Accuracy

As demonstrated in section 7.2.2, numerical instabilities become notably significant in parallel scan algorithms when dealing with a large number of elements. In Figures 7.9 and 7.10, we show RMSE curves on a log-log scale, with $N = 10^6$ and $N = 10^8$, respectively. We find that with $N = 10^6$ the samplers are mostly unbiased, where we observe the bend only after 10^9 samples, which we believe to be caused by unavoidable floating-point inaccuracies. When increasing the amount of weights to $N = 10^8$, the bias of the sampler increases significantly; we therefore suggest keeping the distribution size reasonably small to ensure unbiased sampling.

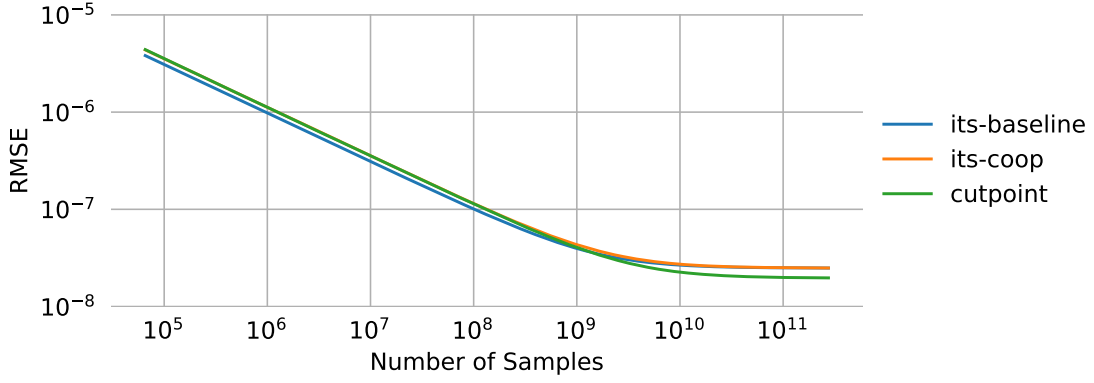


Figure 7.9: RMSE curve of the inverse transform sampling methods with $N = 10^6$.

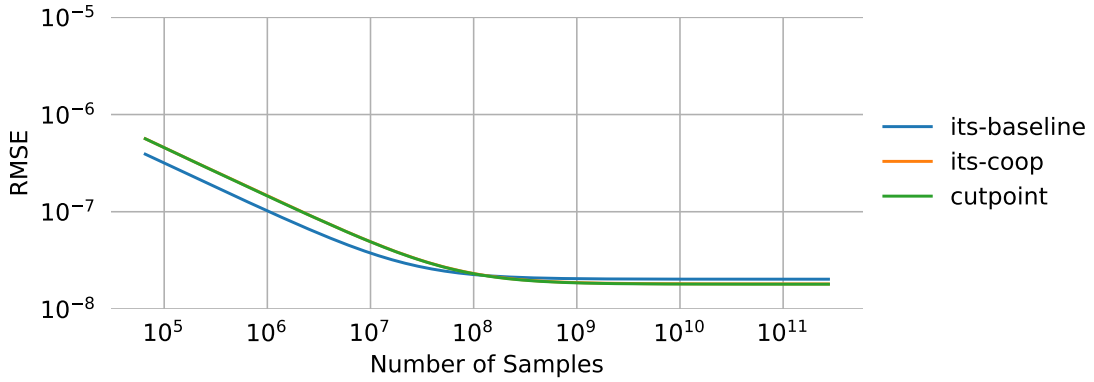


Figure 7.10: RMSE curve of the inverse transform sampling methods with $N = 10^8$.

7.4 Alias method

7.4.1 Parallel splitting based alias table construction

We discuss our PSA implementation in section 6.1. The first step of the PSA algorithm is the computation of the average weight; we find that performing a workgroup local reduction and aggregating the result with floating-point atomics performs best, reaching optimal memory throughput consistently. The second step involves a `scan-partition` operation; we select the best-performing method from section 7.2.1.

In this section, we focus on the evaluation of the splitting and packing steps and determine optimal configurations for both. We initially evaluate them separately. Afterwards, we evaluate combinations of splitting and packing methods to select an optimal configuration for a given distribution size. We generate all inputs (i.e. weights) directly on the device and flush the L2 cache explicitly. Crucially, we retain the L2 cache between PSA steps, resulting in measurements that exhibit performance akin to that observed in the complete implementation.

7.4.1.1 Splitting

We first evaluate the performance of the splitting step depending on the split size. For this, we measure the latency of the split step, with different split sizes, while keeping the amount of splits fixed. Figure 7.11 shows the throughput of the splitting step, depending on the split size. We find that smaller split sizes perform significantly better. These results are expected, as larger split sizes lead to invocations within a subgroup following more divergent paths along the search tree. We also evaluate the p-array search, which replaces the binary search, while all invocations within a subgroup follow the same path. We don't observe any improvement compared to a simple binary search.

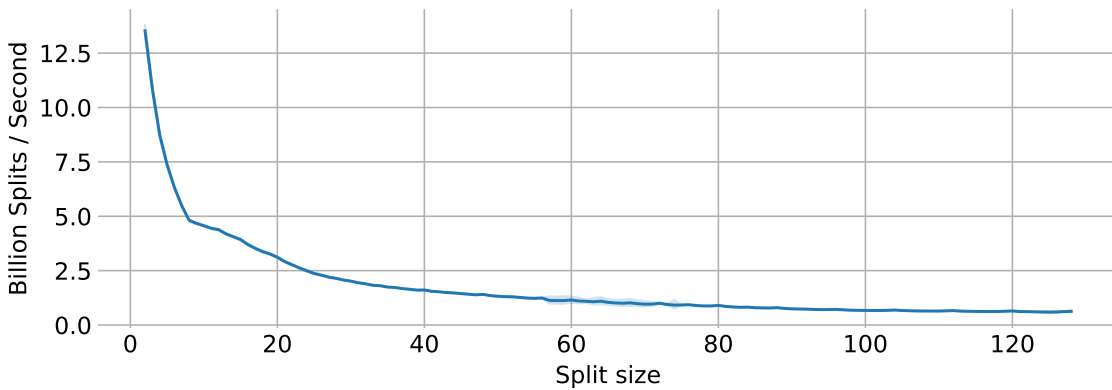


Figure 7.11: Throughput of the split method, depending on the split size. All measurements, where performed with an $K = 10^6$ (i.e. amount of splits).

Secondly, we evaluate the split step, based on the amount of items. Figure 7.12 shows the latency, of splitting with different split sizes. Specifically, we observe that although doubling the split sizes, halves the amount of invocations, it does not reduce the latency in the same way. Instead, we observe that doubling the split size only leads to a reduction of the latency of around 1.5 at best. These results match our initial evaluation of the throughput, where we found that the splitting step reaches the highest throughput with small split sizes.

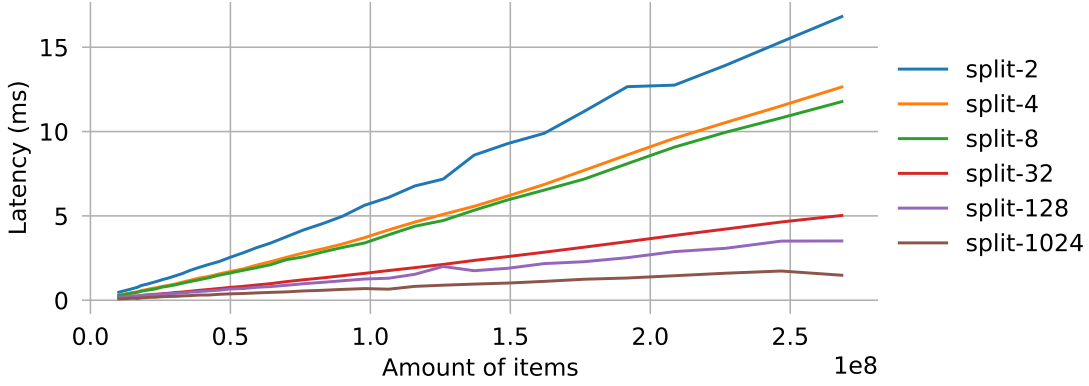


Figure 7.12: Latency of the split methods with varying split sizes, depending on the amount of items (N).

7.4.1.2 Packing

We cover our packing implementation in section 6.1.2, our implementation is parameterized over the size of the subproblems (i.e. the split size) and the amount of invocations that cooperatively process a subproblem. We initially run large parameter sweeps to determine the optimal split size. Figure 7.13 shows the results of these sweeps.

We find that methods, which use few invocations per subproblem, are strongly affected by the split size and only perform well with very small split sizes. On the other side, methods which use many invocations are largely unaffected. This behavior is most likely explained by the improved memory coalescing, when more invocations cooperatively fetch the memory. Nevertheless, allocating more invocations per subproblem also significantly increases the dispatch size. We find that the improvements to the memory coalescing do not decrease the subgroup latency enough to cancel this effect. Moreover, approaches that use few invocations and require small split sizes (resulting in many subproblems) are constrained by the memory bandwidth limit due to a substantial increase in memory accesses to the array of splits. Assigning either 4 or 8 invocations achieves an optimal balance between enhancing memory coalescing, maintaining a compact dispatch size, and minimizing the total memory accesses needed.

Figure 7.14 shows the throughput of the packing methods when selecting the optimal split size, depending on the amount of items. We find that using 4 or 8 invocations per subproblem performs best. Additionally, we observe a significant decrease in throughput at approximately $1.8 \cdot 10^6$ items. This is likely caused by the L2 cache limit, as the combined memory needed for weights, partition indices, prefix sums, splits, and the alias table exceeds the 36MB capacity of the L2 cache, at around $1.8 \cdot 10^6$ items.

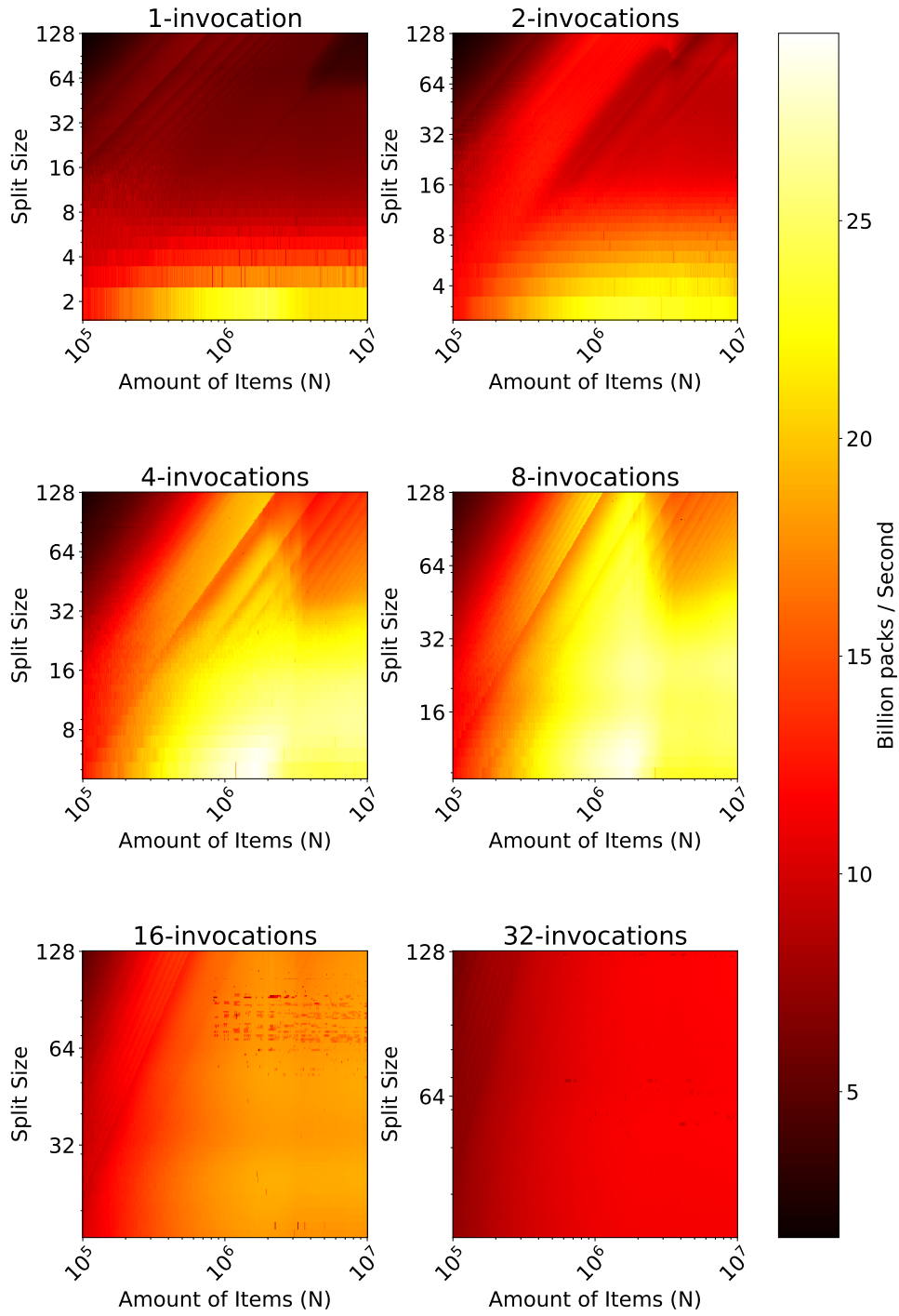


Figure 7.13: Throughput of packing step, depending on the split size and the distribution size.

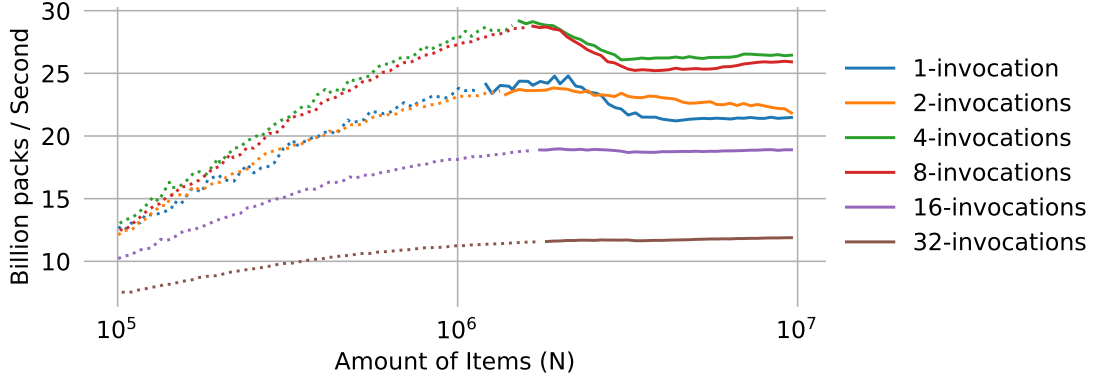


Figure 7.14: Throughput of the packing methods, when selecting the optimal split size from Figure 7.13. Dashed lines represent measurements, where the complete working memory fits into the L2-cache.

7.4.1.3 Split-Packing

After the evaluation of the splitting and packing steps in isolation, we find that both react differently to the subproblem size. The latency of the splitting step decreases linearly with the number of subproblems, whereas the packing step shows a more complex response to various split sizes.

Similar to the previous evaluation, we perform large parameter sweeps to determine the optimal subproblem sizes. Unsurprisingly, smaller split sizes perform worse because they demand more effort during the splitting process. This leads to methods, which allocate fewer invocations per subproblem to perform worse, as they require small split sizes to reach the same throughput. Figure 7.15 shows the total throughput of the splitting and packing steps, when selecting the optimal split size.

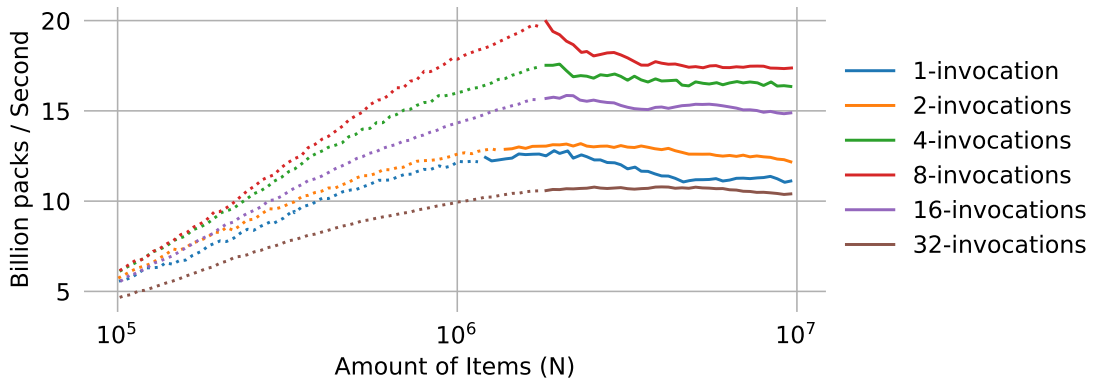


Figure 7.15: Throughput of the splitting and packing step, when selecting the optimal split size. Dashed lines represent measurements, where the complete working memory fits into the L2-cache.

In section 6.1.3, we introduce an optimization, which combines the splitting and packing step into a single compute shader, we refer to this method as **split-packing**. While running the same parameter sweeps for this method (see Figure 7.16), we find that the previously uncompetitive approach of only allocating a single invocation per subproblem shows significantly better performance. Moreover, we find that the previously best performing method, which allocates 8 invocations per subproblem, is further improved. Figure 7.17 shows the throughput of **split-packing** with optimal split sizes.

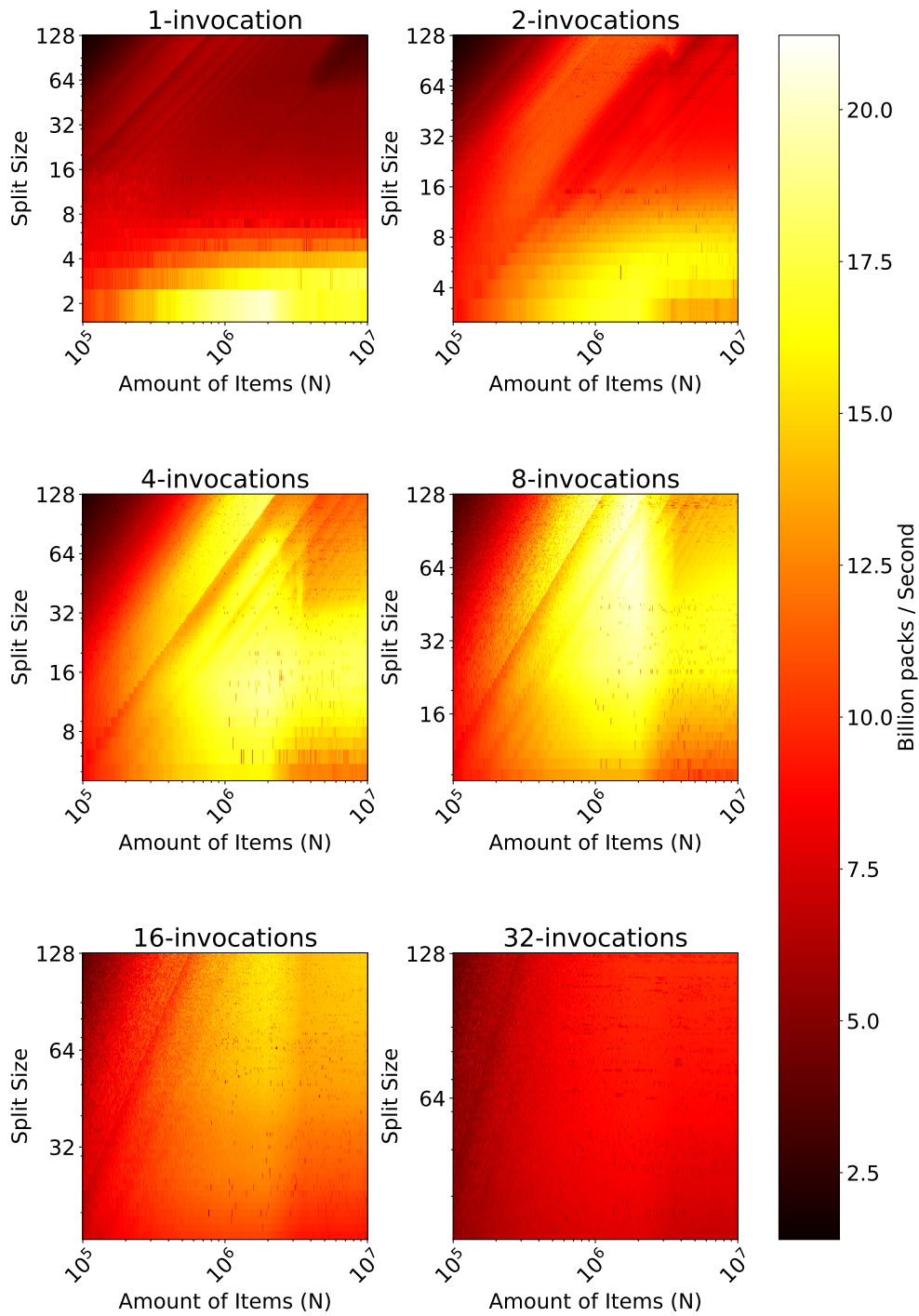


Figure 7.16: Throughput of the `split-packing` method, depending on the amount of items N and the split size (i.e. size of each subproblem). The amount of invocations refers to the amount of invocations which cooperatively perform one pack operation.

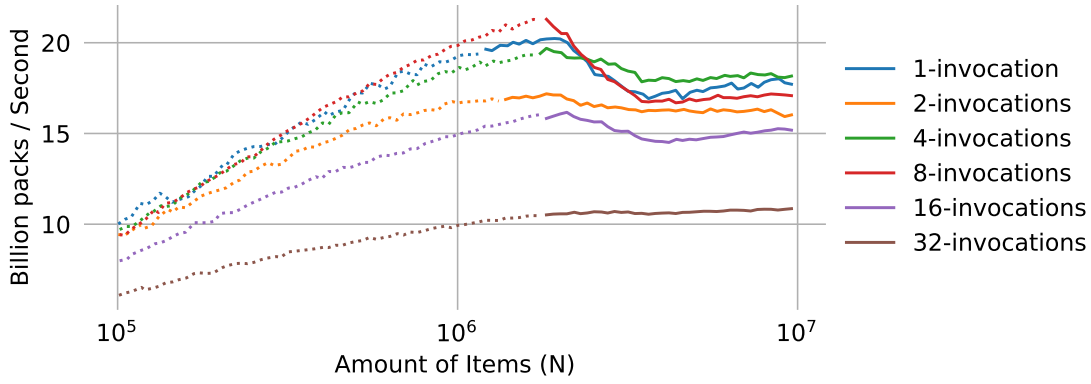


Figure 7.17: Throughput (i.e. amount of alias tables written per second) of the `split-packing` method, with different amounts of invocations allocated per subproblem.

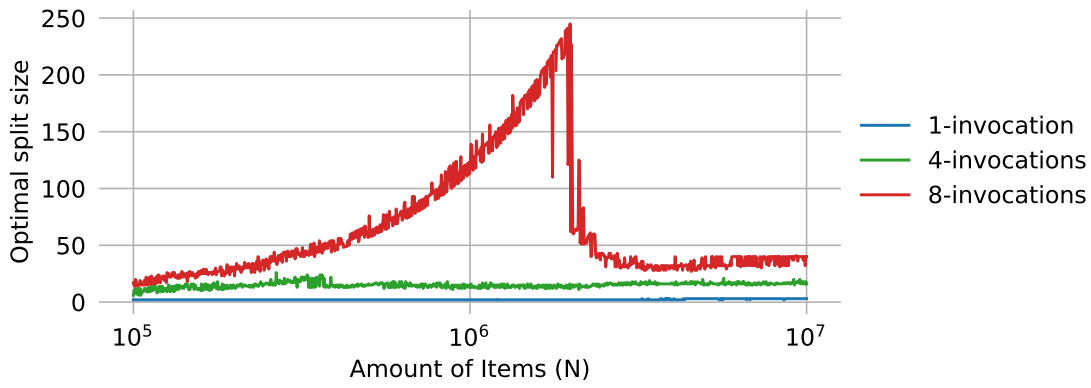


Figure 7.18: Optimal split size of the `split-packing` methods.

Optimal split size

In the following, we analyze the optimal split size for a given configuration. Specifically, we restrict our analysis to the most competitive `split-packing` methods. Configurations, which allocate 1 or 4 invocations per subproblem, perform best with the same split size regardless of the amount of items and regardless of caching effects. As discussed previously, allocating a single invocation only performs well with a split size of 2. We observe a similar pattern for methods which allocate 4 invocations, where they consistently reach the peak throughput with a split size of around 16. This property is especially interesting for applications which frequently perform the PSA algorithm with a different amount of items, as the selection of the optimal split size is trivial and not dependent on the amount of items.

Allocating eight invocations per subproblem can outperform configurations with fewer invocations for certain distribution sizes, however the selection of the optimal split size becomes more complex. Once eight or more invocations are used per subproblem, the splitting process becomes less efficient because multiple invocations in the same subgroup compute identical splits. Consequently, load balancing between the splitting and packing steps becomes crucial in selecting the optimal split size. Smaller splits increase the number of split operations, leading to redundant computation, whereas overly large splits cannot be effectively distributed across SMT-processors and degrade packing efficiency.

For configurations above eight invocations, the best split size strikes a balance between these opposing inefficiencies and ensures full GPU utilization. Another contributing factor

is that GPUs optimize for throughput rather than latency: allocating even one more invocation than the GPU can occupy forces serialization and may double total latency.

Figure 7.19 illustrates how many total invocations are required with eight invocations per subproblem. We typically select the split size so the total invocations remain just under the device’s occupancy limit—as long as all working data fits in the L2 cache. Beyond roughly $1.8 \cdot 10^6$ items, however, the data no longer fits in the L2 cache, so the approach shifts to using a constant split size. We hypothesize this arises because the packing step depends heavily on cached values; once out of cache, performance becomes bound by packing latency. In that scenario, optimizing for the packing step—particularly by maximizing memory coalescing—becomes more beneficial than minimizing the amount of splits.

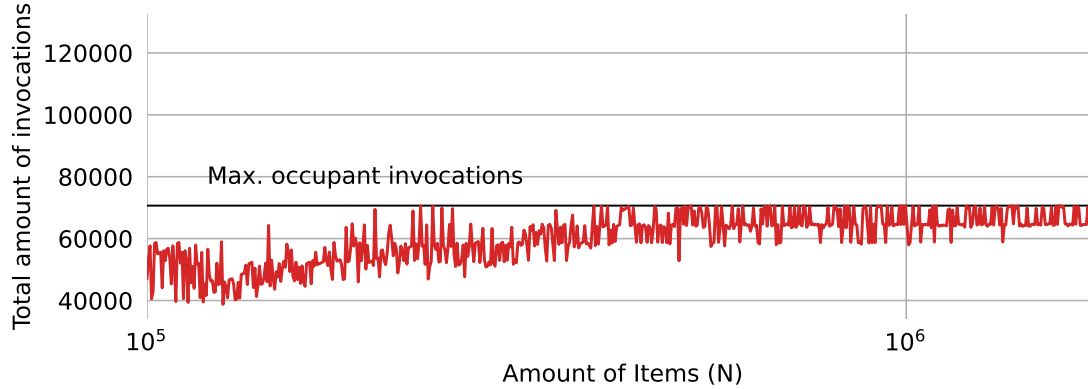


Figure 7.19: Amount of invocations required to perform the complete `split-packing` step, when selecting the optimal split size. The working memory entirely fits within the L2 cache for all measurements presented here.

7.4.1.4 Complete construction

Finally, we evaluate the complete construction, involving the computation of the average weight, the `partition-scan` and the `split-packing` steps.

We first test, writing the partition elements during the `scan-partition` step and using them instead of the weights array in the `split-packing` procedure. As mentioned in section 7.2.1, we find that this increases the overall latency of the `scan-partition` procedure by around 80%. Although reading the partition elements instead of the weights array improves the memory coalescing in the `split-packing` step, it does not improve the latency by enough to justify the additional overhead during `scan-partition`. We find that writing the partitions performs 5% worse than reading the weights directly.

Furthermore, we examine our PSA implementation with different weight distributions. Specifically, we evaluate the following weight distributions:

- **Uniform:** Uniformly distributed weights between 0 and 1.
- **Normal:** Normally distributed weights, specifically we generate independent samples with a standard normal distribution and use the absolute value as a weight.
- **Exponential:** Exponentially distributed weights with a exponent of 1.
- **Sorted:** We generate uniformly distributed weights and sort the resulting weights.
- **Brownian Noise:** Brownian noise can be generated by integrating uniformly distributed numbers. We generate Brownian noise with a leaky integrator (sometimes referred to as Ornstein–Uhlenbeck noise) and use it as a weight. We specifically provide Brownian noise, as it leads to consecutive weights being dependent.

We find that the performance of our PSA implementation is mostly unaffected by the distribution of weights. The sole exception occurs when the weights are fully ordered, resulting in our PSA implementation being approximately 5% quicker.

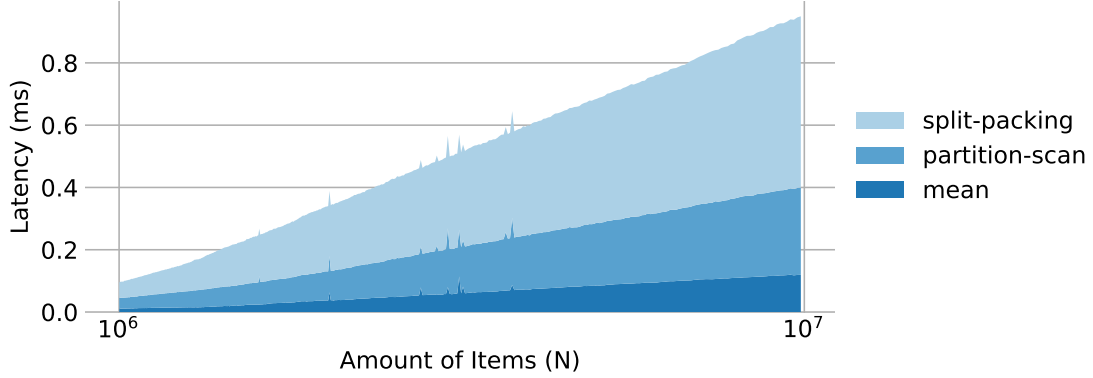


Figure 7.20: Latency of the individual steps of the PSA algorithm. For all measurements we select the optimal configuration based on large parameter sweeps.

7.4.2 Sampling

We cover our sampling implementations in section 6.2. Figure 7.21 shows the sampling throughput, with varying distribution sizes. The baseline method draws a single independent sample with every invocation. For the `sectioned-sampling` strategy, we arbitrarily select a section size of 128 items. Similar to the evaluation of the inverse transform method, we find that restricting the sample space within a subgroup can greatly increase the sample throughput. In Figure 7.21, we show both measurements with and without flushing the L2 cache between the construction and the sampling process. Measurements where the L2 is not flushed represent sampling directly after the construction.

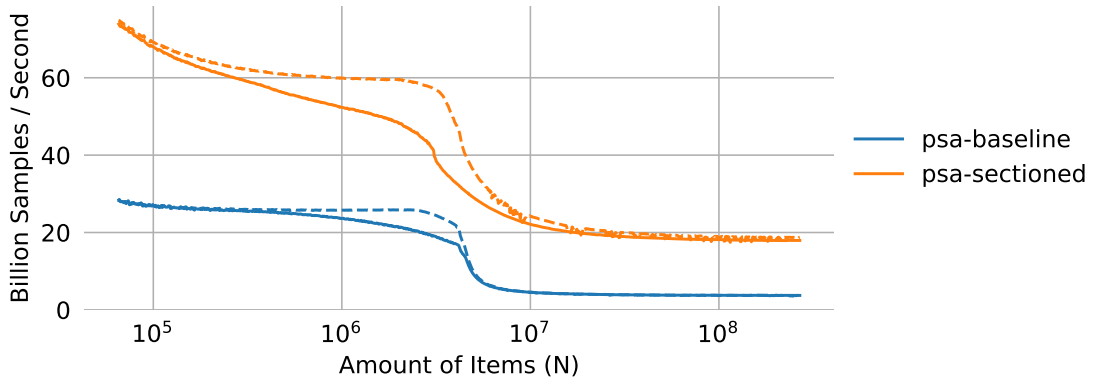


Figure 7.21: Sampling throughput of the alias-method, when drawing 10^7 samples.

Accuracy

We conduct short tests to determine the bias of the alias method. Figure 7.22 and 7.23 show the RMSE on a log-log scale. For both $N = 10^6$ and $N = 10^8$, we find that the alias method is mostly unbiased, where the bend is most likely explained by the floating-point inaccuracies described in section 6.1.1.

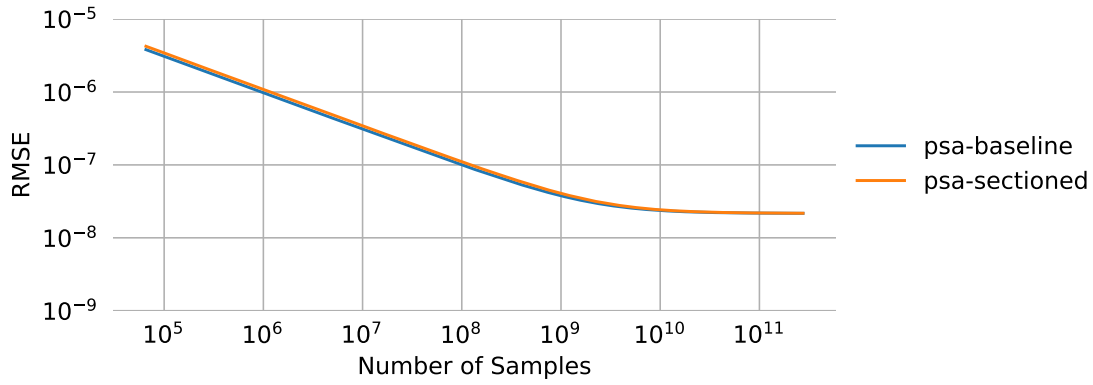


Figure 7.22: RMSE curve of the alias-method with $N = 10^6$

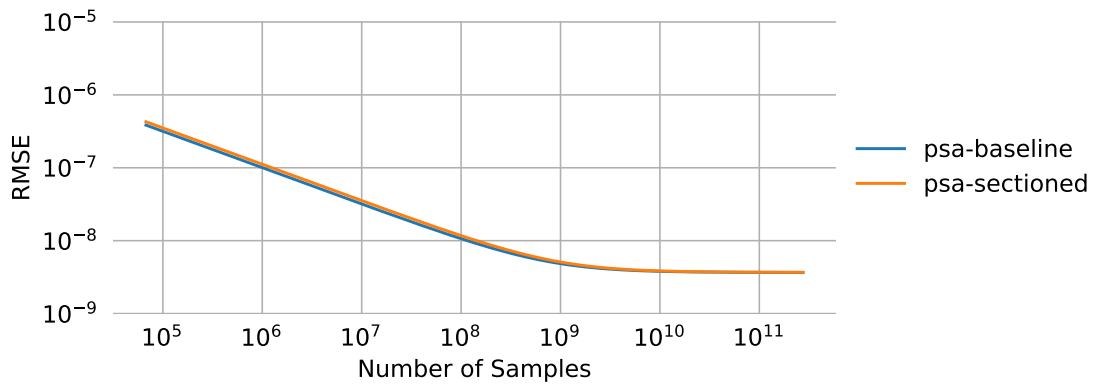


Figure 7.23: RMSE curve of the alias-method with $N = 10^8$

7.5 Comparison

In the subsequent section, we conduct a comparison of the performance of our weighted random samplers. We specifically assess two scenarios: The first scenario pertains to applications such as sampling a complex material BRDF, in which a single set of weights is repeatedly sampled. In this instance, we can largely disregard the construction time and concentrate solely on achieving high sampling throughput or low RMSE after a set time. The second scenario takes into account the construction time and is most relevant when a set of weights is sampled only once.

Figure 7.24 shows the sampling throughput of our implementations when ignoring construction. As expected, we find that the alias method with sectioned sampling consistently performs best. Furthermore, the cutpoint method unexpectedly achieves a very high average throughput that is 85% of that attained by the alias method, which is impressive as it requires significantly less work during construction. Our baseline implementations, which neither use sectioned nor cooperative sampling, perform considerably worse. Nevertheless, these methods are unique in producing independent samples.

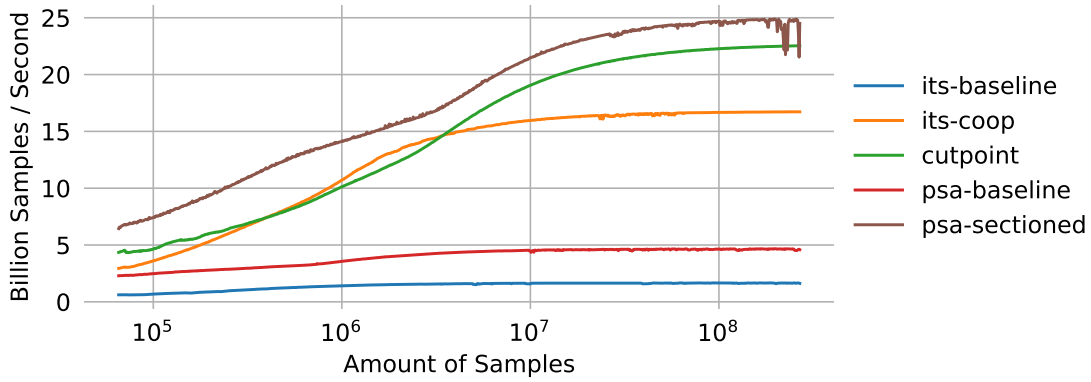


Figure 7.24: Comparison of the sampling throughputs of the proposed methods, with $N = 10^7$. If applicable we use a `section-size` of 128.

In Figure 7.25, we show the speed at which the random samplers converge. Specifically, it shows the Root Mean Square Error, based on the sampling latency. We find that although the baseline methods generally lead to a lower RMSE with a fixed sample size, when we additionally consider the much higher sampling throughput of sectioned or cooperative sampling, we find that those methods converge significantly faster.

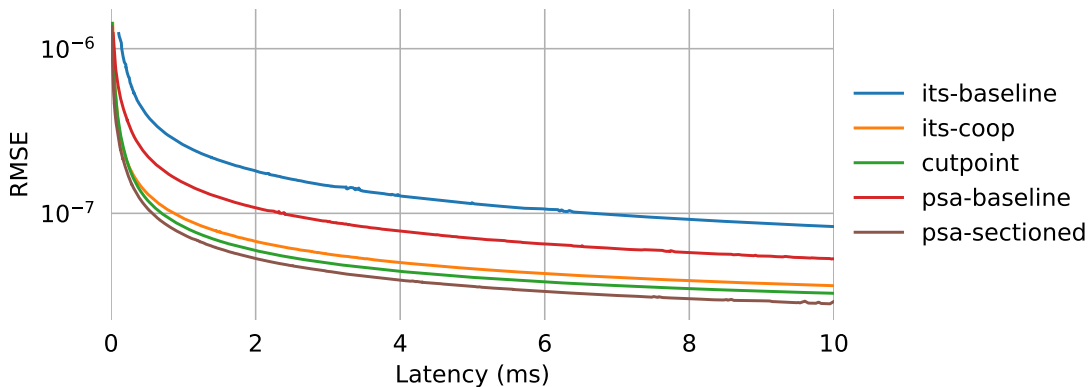


Figure 7.25: Rate of convergence of the sampling methods with $N = 10^7$, while only accounting for the sampling time.

After the evaluation of the rate of convergence, when only considering the sampling time, we continue with the same evaluation, while considering both construction and sampling time. Figure 7.26 shows the speed at which the random samplers converge when we include the construction time. As expected, we find that the cutpoint and inverse transform method perform best with little time, because of their fast construction time. Moreover, we find that after the time required to construct the alias table, the alias method catches up relatively quickly.

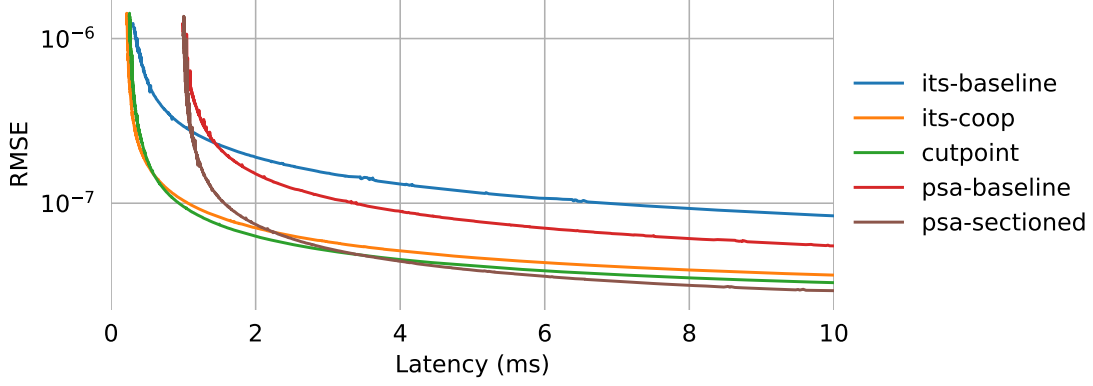


Figure 7.26: Rate of convergence of the sampling methods with $N = 10^7$, when considering both the construction and sampling time.

The point at which the alias method overtakes is dependent on the amount of weights. In the following, we perform large parameter sweeps to select the optimal sampling strategy for a given amount of items and samples drawn.

Figure 7.27 shows the method that reaches the highest sample throughput when considering both the construction and sampling time. Importantly, Figure 7.27 does not consider the quality of the samples. Figure 7.28 selects the optimal method based on the RMSE. Specifically, for a given distribution size and time constraint, it selects the method that achieves the lowest RMSE.

We find that, when only considering the sampling throughput, the cutpoint and inverse transform sampling with cooperative narrowing perform best in most cases, which is expected as they require very little work during construction. Importantly, we find that the alias method is only best if the amount of samples drawn is significantly larger than the amount of items. When we now additionally consider the rate of convergence (Figure 7.28), we observe that for smaller distribution sizes, the alias method becomes more competitive; however, the cutpoint method still outperforms the alias method if the amount of items is larger than 10^7 due to its very fast construction.

In general, we conclude that for applications, which require sampling a set of weights only once, the alias-method is applicable only if the amount of samples is around one order of magnitude higher than the amount of items; in all other cases, much simpler sampling strategies such as the cutpoint method can perform significantly better.

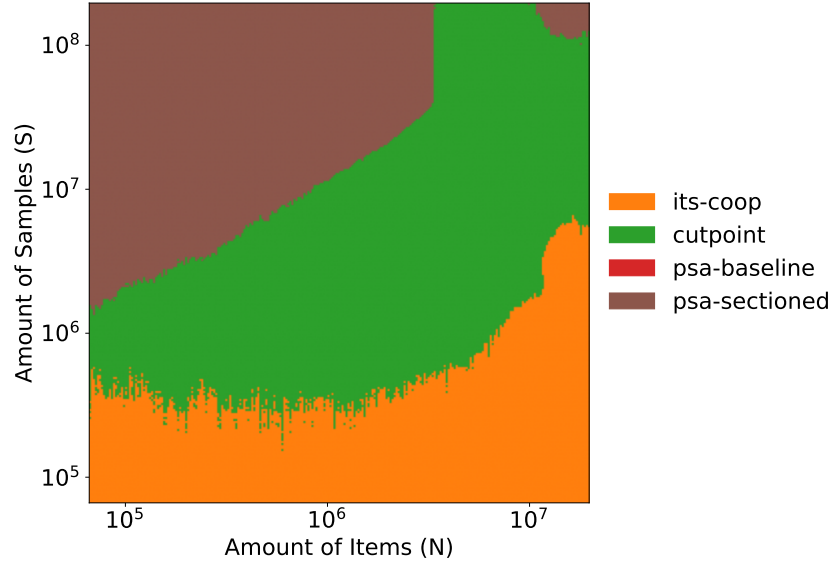


Figure 7.27: Method, which achieves the highest sample throughput for a given amount of items/samples.

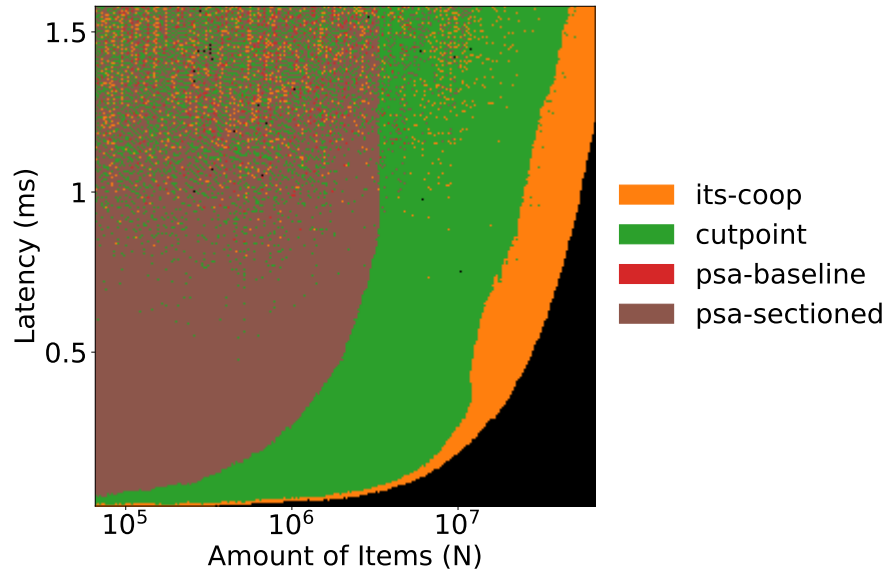


Figure 7.28: Method, which achieves the lowest RMSE, in a given time. Black pixels indicate that no matching measurement exists. The noise in the upper left corner is only because not enough measurements exists for that region, it does not indicate that all methods perform equal.

8. Conclusion

We present an efficient and portable implementation of weighted random sampling with the alias method, based on the PSA algorithm developed by Hübschle-Schneider and Sanders. We provide a portable implementation of a single dispatch parallel partition and parallel scan algorithm, which performs very close to the memory bandwidth limit and only relies on guarantees provided by the Vulkan memory model. We further introduce optimizations to the splitting and packing step, which avoid redundant writes to global memory and improve the memory bandwidth utilization. Compared to previous CUDA implementations of PSA [LHS21], we are able to achieve a speedup of 2.5. To facilitate efficient sampling on the GPU, we implement sectioned sampling strategies for inverse transform sampling, the cutpoint method and the alias method, which significantly increase the sample throughput, at the cost of higher sampling variance.

Future Work

We do not address the weak serial dependency between the `scan-partition` and `split-packing` step (see section 6.1). Hübschle-Schneider and Sanders presents `PSA+`, which greedily packs a portion of the alias table entries during the partition step, and because we do not provide such an implementation, it remains unclear how `PSA+` affects the performance of our splitting and packing implementations.

Further, we expect that using block-wise scans instead of complete prefix sums over the heavy and light elements can potentially improve performance and numerical stability of our PSA implementation.

Bibliography

- [BK82] Brent and Kung, “A Regular Layout for Parallel Adders”, *IEEE Transactions on Computers*, vol. C-31, no. 3, pp. 260–264, Mar. 1982, Conference Name: IEEE Transactions on Computers, ISSN: 1557-9956. DOI: 10.1109/TC.1982.1675982. [Online]. Available: <https://ieeexplore.ieee.org/document/1675982/?arnumber=1675982> (visited on 03/06/2025).
- [CA74] H.-C. Chen and Y. Asau, “On Generating Random Variates from an Empirical Distribution”, en, *A I I E Transactions*, vol. 6, no. 2, pp. 163–166, Jun. 1974, ISSN: 0569-5554. DOI: 10.1080/05695557408974949. [Online]. Available: <http://www.tandfonline.com/doi/abs/10.1080/05695557408974949> (visited on 03/26/2025).
- [Cor17] N. Corporation, *NVIDIA TESLA V100 GPU Architecture*, Aug. 2017. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf> (visited on 10/27/2024).
- [Dua16] M. G. Duane Merrill, *Single-pass Parallel Prefix Scan with Decoupled Look-back | Research*, Mar. 2016. [Online]. Available: https://research.nvidia.com/publication/2016-03_single-pass-parallel-prefix-scan-decoupled-look-back (visited on 10/24/2024).
- [GSO12] K. Gupta, J. A. Stuart, and J. D. Owens, “A study of Persistent Threads style GPU programming for GPGPU workloads”, in *2012 Innovative Parallel Computing (InPar)*, May 2012, pp. 1–14. DOI: 10.1109/InPar.2012.6339596. [Online]. Available: <https://ieeexplore.ieee.org/document/6339596> (visited on 10/24/2024).
- [HS05] W. D. Hillis and G. L. Steele, “Data parallel algorithms”, en, 2005.
- [HS22] L. Hübschle-Schneider and P. Sanders, “Parallel Weighted Random Sampling”, *ACM Trans. Math. Softw.*, vol. 48, no. 3, 29:1–29:40, Sep. 2022, ISSN: 0098-3500. DOI: 10.1145/3549934. [Online]. Available: <https://doi.org/10.1145/3549934> (visited on 10/23/2024).
- [KS73] P. M. Kogge and H. S. Stone, “A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations”, *IEEE Transactions on Computers*, vol. C-22, no. 8, pp. 786–793, Aug. 1973, Conference Name: IEEE Transactions on Computers, ISSN: 1557-9956. DOI: 10.1109/TC.1973.5009159. [Online]. Available: <https://ieeexplore.ieee.org/document/5009159/?arnumber=5009159> (visited on 03/06/2025).
- [Lev21] R. Levien, *Prefix sum on portable compute shaders*, en, Nov. 2021. [Online]. Available: <https://raphlinus.github.io/gpu/2021/11/17/prefix-sum-portable.html> (visited on 10/23/2024).

- [LHS21] H.-P. Lehmann, L. Hübschle-Schneider, and P. Sanders, “Weighted Random Sampling on GPUs”, *ArXiv*, Jun. 2021. [Online]. Available: <https://www.semanticscholar.org/paper/Weighted-Random-Sampling-on-GPUs-Lehmann-H%C3%BCbschle-Schneider/177423a8617187754bc291fd83908ced2ad5a398> (visited on 10/23/2024).
- [Lum13] J. Lumbroso, *Optimal Discrete Uniform Generation from Coin Flips, and Applications*, en, arXiv:1304.1916 [cs], Apr. 2013. DOI: 10.48550/arXiv.1304.1916. [Online]. Available: <http://arxiv.org/abs/1304.1916> (visited on 03/19/2025).
- [NVI] NVIDIA, *GPU Gems 3 Chapter 39. Parallel Prefix Sum (Scan) with CUDA*, en-US. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda> (visited on 02/02/2025).
- [SDB+16] T. Sorensen, A. F. Donaldson, M. Batty, G. Gopalakrishnan, and Z. Rakić, “Portable inter-workgroup barrier synchronisation for GPUs”, in *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, ser. OOPSLA 2016, New York, NY, USA: Association for Computing Machinery, Oct. 2016, pp. 39–58, ISBN: 978-1-4503-4444-9. DOI: 10.1145/2983990.2984032. [Online]. Available: <https://dl.acm.org/doi/10.1145/2983990.2984032> (visited on 10/24/2024).
- [SHG] S. Sengupta, M. Harris, and M. Garland, “Efficient Parallel Scan Algorithms for GPUs”, en,
- [Skl60] J. Sklansky, “Conditional-Sum Addition Logic”, *IRE Transactions on Electronic Computers*, vol. EC-9, no. 2, pp. 226–231, Jun. 1960, Conference Name: IRE Transactions on Electronic Computers, ISSN: 0367-9950. DOI: 10.1109/TEC.1960.5219822. [Online]. Available: <https://ieeexplore.ieee.org/document/5219822/?arnumber=5219822> (visited on 03/06/2025).
- [SMDS11] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, “Parallel random numbers: As easy as 1, 2, 3”, en, in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, Seattle Washington: ACM, Nov. 2011, pp. 1–12, ISBN: 978-1-4503-0771-0. DOI: 10.1145/2063384.2063405. [Online]. Available: <https://dl.acm.org/doi/10.1145/2063384.2063405> (visited on 03/19/2025).
- [SSR+21] T. Sorensen *et al.*, “Specifying and testing GPU workgroup progress models”, en, *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–30, Oct. 2021, ISSN: 2475-1421. DOI: 10.1145/3485508. [Online]. Available: <https://dl.acm.org/doi/10.1145/3485508> (visited on 10/24/2024).
- [Vos91] M. Vose, “A linear algorithm for generating random numbers with a given distribution”, *IEEE Transactions on Software Engineering*, vol. 17, no. 9, pp. 972–975, Sep. 1991, Conference Name: IEEE Transactions on Software Engineering, ISSN: 1939-3520. DOI: 10.1109/32.92917. [Online]. Available: <https://ieeexplore.ieee.org/document/92917> (visited on 10/23/2024).
- [Wal77] A. J. Walker, “An Efficient Method for Generating Discrete Random Variables with General Distributions”, *ACM Trans. Math. Softw.*, vol. 3, no. 3, pp. 253–256, Sep. 1977, ISSN: 0098-3500. DOI: 10.1145/355744.355749. [Online]. Available: <https://dl.acm.org/doi/10.1145/355744.355749> (visited on 10/23/2024).